P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

INTERNSHIP / UDP / TRAINING

FINAL PROJECT REPORT

(Hard-Bound Project Report Format)

Particular	Details
Project Title	PromptForm AI – An AI-Powered Intelligent Form Generation and Analytics Platform
Student Name	[STUDENT NAME]
Enrollment Number	[ENROLLMENT NUMBER]
Program / Branch	[B.Tech – Computer Engineering]
Semester	[SEMESTER]
Company / Organization	[COMPANY / ORGANIZATION NAME]
Department / Domain	Full-Stack Web Development / Applied Generative AI
Mentor (Company)	[COMPANY MENTOR NAME]
Mentor (Institute)	[INSTITUTE MENTOR NAME]
Academic Year	2026–2027

CERTIFICATE

OF COMPLETION

This is to certify that

[STUDENT NAME]

Enrollment No.: [ENROLLMENT NUMBER]

of [B.Tech – Computer Engineering], [SEMESTER], has successfully completed the Internship / UDP / Training project titled

"PromptForm AI – An AI-Powered Intelligent Form Generation and Analytics Platform"

during the period from [START DATE] to [END DATE] at [COMPANY / ORGANIZATION NAME], under the guidance of [COMPANY MENTOR NAME] and [INSTITUTE MENTOR NAME].

The project work, final implementation, testing and validation, documentation, and demonstration were completed as required for the Internship / UDP / Training and found satisfactory.

Date of Issue: [DD / MM / YYYY]

_____ _____

Mentor (Company) **Mentor (Institute)**

DECLARATION

I hereby declare that this report is a consolidated and authentic record of the Internship / UDP / Training project work titled "PromptForm AI – An AI-Powered Intelligent Form Generation and Analytics Platform" carried out by me during the period [START DATE] to [END DATE] at [COMPANY / ORGANIZATION NAME]. All sources used have been appropriately acknowledged and cited.

I further declare that the work presented in this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. Third-party libraries, frameworks, services and published works used in the project are listed in Chapter 15 (References) and cited in the text using the APA 7th Edition style.

Date: [DD / MM / YYYY]

Place: Surat, Gujarat

_____

[STUDENT NAME]

Enrollment No.: [ENROLLMENT NUMBER]

ACKNOWLEDGEMENT

I express my sincere gratitude to [COMPANY / ORGANIZATION NAME] for giving me the opportunity to carry out the project “PromptForm AI – An AI-Powered Intelligent Form Generation and Analytics Platform” as part of my Internship / UDP / Training, and for providing the development environment, tools and real-world problem context that shaped this work.

I am deeply thankful to my company mentor, [COMPANY MENTOR NAME], for continuous technical guidance, code reviews and practical advice on architecture, deployment and product quality throughout the project. Their feedback during each progress review helped convert an initial prototype into a production-ready platform.

I am equally grateful to my institute mentor, [INSTITUTE MENTOR NAME], School of Engineering, P P Savani University, for academic supervision, for reviewing the four progress reports, and for valuable suggestions on documentation, testing methodology and report presentation.

I thank the Head of the Department, the faculty members of the School of Engineering and the Training and Placement cell of P P Savani University for the structured internship framework, timely evaluations and constant encouragement.

I also acknowledge the open-source communities behind Next.js, React, Express, Prisma, PostgreSQL and the Google Gemini API, whose documentation and tools made this project possible. Finally, I thank my family and friends for their patience, motivation and support during the entire internship period.

ABSTRACT / EXECUTIVE SUMMARY

Online forms are the most common way for organisations to collect registrations, feedback, quiz answers, patient intake details and survey data. Existing form builders such as Google Forms, Microsoft Forms, Typeform and Jotform require the author to add every field manually, choose input types, write options, configure validation and design the theme. For a typical 8–10 question form this takes 15–30 minutes and frequently produces inconsistent field types, missing validations and poor respondent experience. Paper forms that already exist in an organisation must also be retyped by hand.

This project presents **PromptForm AI**, a full-stack Software-as-a-Service (SaaS) platform that generates complete, editable and shareable web forms from a natural-language prompt or from an uploaded document (PDF, DOCX, spreadsheet or image). The objectives were to reduce form creation time to seconds, select correct field types and validations automatically, support quizzes with scoring and anti-cheat controls, and provide response analytics and team collaboration in one product.

The system follows a three-tier architecture developed in a TypeScript monorepo using an Agile, sprint-based methodology. The presentation tier is built with Next.js 15, React 19, Tailwind CSS and Zustand; the application tier is an Express 4 REST API secured by JWT authentication, bcrypt password hashing, Helmet, CORS and rate limiting; the data tier uses PostgreSQL through Prisma ORM with 16 relational models. Generation is handled by a hybrid engine: a deterministic rule-based “AIFormBrain” (intent, language, domain and industry detection, field intelligence, quiz engine and quality reviewer) combined with Google Gemini large language models through a five-model fallback chain with JSON-schema output.

The platform was verified with 50 automated test cases — 39 platform QA tests covering authentication, forms, submissions, teams, analytics and security, and 11 AI-generation tests covering text, quiz, survey, PDF and image inputs, credit limits and plan limits. All tests passed after nine defects identified during QA were fixed and re-verified. A form that takes about 20 minutes to build manually is produced by PromptForm AI in under 10 seconds, published immediately with a unique share link, and can be refined in a drag-and-drop visual builder.

The project demonstrates that combining deterministic domain rules with LLM generation yields reliable, structured output suitable for production use. The platform

was deployed on Vercel (frontend) and Render (backend) with a managed PostgreSQL database, and it provides a strong foundation for future features such as payments, conditional-logic generation and multilingual voice input.

Keywords: Generative AI, Form Builder, Large Language Models, Google Gemini, Next.js, Express, Prisma ORM, PostgreSQL, SaaS, Prompt Engineering

TABLE OF CONTENTS

Chapter / Section No.	Title	Tentative Page
Preliminary Pages		
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract / Executive Summary	5
	Table of Contents	7
	List of Figures	11
	List of Tables	13
	List of Abbreviations / Acronyms	15
Chapter 1	**Introduction**	17
1.1	Background of the Project	17
1.2	Problem Statement	17
1.3	Need / Motivation	18
1.4	Objectives	18
1.5	Scope of the Project	19
1.6	Expected Outcomes vs Final Outcomes	20
1.7	Report Organization	20
Chapter 2	**Organization / Internship / Training Profile**	22
2.1	Organization / Company Overview	22
2.2	Department / Domain	22
2.3	Role and Responsibilities	22
2.4	Development / Work Environment	23
2.5	Internship Details	23
Chapter 3	**Literature Review / Background Study**	25
3.1	Domain Background	25
3.2	Existing Approaches / Related Work	26
3.3	Comparative Analysis	26
3.4	Research Gap / Project Justification	27
Chapter 4	**Requirements Analysis and Project Scope**	28
4.1	Functional Requirements	28
4.2	Non-Functional Requirements	29

Chapter / Section No.	Title	Tentative Page
4.3	Hardware Requirements (if applicable)	30
4.4	Software Requirements	30
4.5	Data / Dataset Requirements (if applicable)	31
4.6	Constraints and Assumptions	31
Chapter 5	**Methodology and Project Planning**	33
5.1	Final Development / Research Methodology	33
5.2	Project Workflow	33
5.3	Project Phases and Deliverables	34
5.4	Progress Report Consolidation	34
5.5	Project Schedule / Timeline	35
Chapter 6	**System Analysis and Design**	36
6.1	Existing System / Current Practice	36
6.2	Proposed System	36
6.3	System Architecture	37
6.4	Workflow / Process Flow	37
6.5	UML / Design Diagrams	38
6.6	Module-wise Design	41
6.7	Database / Data Design	42
6.8	Interface / UI Design (if applicable)	43
Chapter 7	**Technology Stack and Development Environment**	45
7.1	Programming Languages	45
7.2	Frameworks / Libraries / APIs	45
7.3	Database / Storage	46
7.4	Hardware / Cloud / Deployment Environment	46
7.5	Development Tools	47
7.6	Technology Selection Rationale	47
Chapter 8	**System Implementation**	48
8.1	Implementation Overview	48
8.2	Module / Feature 1 – AI Form Generator	48
8.3	Module / Feature 2 – Visual Form Builder	49
8.4	Module / Feature 3 – Responses, Quiz Engine and Analytics	50
8.5	Algorithm / Logic / Model Description	51
8.6	API / External Service Integration	53

Chapter / Section No.	Title	Tentative Page
8.7	Database Implementation	54
8.8	Security / Access Control (if applicable)	54
8.9	Integration of Modules	55
Chapter 9	**Testing and Validation**	56
9.1	Testing Strategy	56
9.2	Unit / Module Testing	56
9.3	Functional Testing	57
9.4	Integration Testing	58
9.5	System Testing	59
9.6	Performance Testing	60
9.7	User / Acceptance Testing	60
9.8	Experimental Validation (if applicable)	61
9.9	Defects / Bugs and Resolution	61
9.10	Final Validation Statement	62
Chapter 10	**Results and Performance Analysis**	63
10.1	Final Results	63
10.6	Results Discussion	63
10.2	Objective-wise Achievement	64
10.3	Performance Metrics	64
10.4	Comparative Results (if applicable)	65
10.5	Final Output Screenshots	65
Chapter 11	**Challenges, Solutions and Improvements**	68
11.1	Major Challenges	68
11.2	Technical Challenges	68
11.3	Process / Coordination Challenges	69
11.5	Lessons from Challenges	69
11.4	Improvements Made During Development	69
Chapter 12	**Individual Contribution and Professional Learning**	71
12.1	Individual Contribution	71
12.2	Technical Skills Acquired	71
12.3	Professional Skills Acquired	72
12.4	Major Learning Outcomes	72
Chapter 13	**Limitations and Future Scope**	73

Chapter / Section No.	Title	Tentative Page
13.1	Current Limitations	73
13.2	Future Scope	73
13.3	Scalability / Extension Opportunities	74
Chapter 14	**Conclusion**	75
14.1	Final Project Status	75
Chapter 15	**References**	77
15.1	References	77
15.2	Citation Requirements	78
Chapter A	**Appendices**	79
Appendix A	Progress Report 1 Summary	79
Appendix B	Progress Report 2 Summary	79
Appendix C	Progress Report 3 Summary	79
Appendix D	Reporting 4 / Final Evaluation	79
Appendix E	Additional Screenshots / Outputs	80
Appendix F	Certificates / Internship Evidence	85
Appendix G	Source Code / Repository / Deployment Evidence	85
Appendix H	Mentor / Evaluator Remarks	90
Chapter B	**Final Hard-Bound Submission Checklist**	91
Chapter C	**Formatting and Submission Guidelines**	92

LIST OF FIGURES

Figure No.	Title	Page
Figure 5.1	Overall Project Workflow	33
Figure 5.2	Project Schedule (Gantt Chart)	35
Figure 6.1	Final System Architecture	37
Figure 6.2	Final Workflow / Process Flow	38
Figure 6.3	Major UML / ER / Design Diagram — Use Case Diagram	39
Figure 6.4	Data Flow Diagram (Level 1)	39
Figure 6.5	Class Diagram — AI Engine and Services	40
Figure 6.6	Sequence Diagram — AI Form Generation	40
Figure 6.7	Activity Diagram — Form Submission	41
Figure 6.8	ER Diagram / Database Schema	41
Figure 6.9	Major Interface / UI Design	44
Figure 8.1	Module 1 Implementation / Output	49
Figure 8.2	Module 2 Implementation / Output	50
Figure 8.3	Module 3 Implementation / Output	51
Figure 8.4	Authentication and Authorization Flow	55
Figure 9.1	Test Cases Passed by Suite	62
Figure 10.1	Final Output / Result Screenshot 1 (Workspace Overview Dashboard)	66
Figure 10.2	Final Output / Result Screenshot 2 (Public Responder)	66
Figure 10.3	Final Output / Result Screenshot 3 (Responses Ledger)	67
Figure 10.4	Final Output / Result Screenshot 4 (Template Marketplace)	67
Figure A.1	Supporting Evidence — Login Page	80
Figure A.2	Supporting Evidence — Form Creation Time Comparison	80
Figure A.3	Supporting Evidence — Registration Page	81
Figure A.4	Supporting Evidence — Pricing Plans	81
Figure A.5	Supporting Evidence — Forms Library	82
Figure A.6	Supporting Evidence — Workspace Analytics	82
Figure A.7	Supporting Evidence — Team Workspace	83
Figure A.8	Supporting Evidence — Account Settings	83
Figure A.9	Supporting Evidence — Billing and Credits	84
Figure A.10	Supporting Evidence — Module Architecture	84
Figure A.11	Supporting Evidence — Deployment Diagram	85

Figure No.	Title	Page
Figure A.12	Supporting Evidence — Landing Page (full length)	89

LIST OF TABLES

Table No.	Title	Page
Table 1.1	Expected outcomes compared with final outcomes	20
Table 1.2	Scope Matrix	21
Table 2.1	Internship details	23
Table 3.1	Review of existing approaches and related work	26
Table 3.2	Comparative analysis of existing approaches and the proposed project	26
Table 4.1	Functional requirements	28
Table 4.2	Non-functional requirements	29
Table 4.3	Hardware requirements	30
Table 4.4	Software requirements	30
Table 4.5	Data requirements	31
Table 5.1	Project phases and deliverables	34
Table 5.2	Progress report consolidation	34
Table 5.3	Project schedule – planned vs actual	35
Table 6.1	Module-wise design	41
Table 6.2	Database entities	42
Table 7.1	Programming languages	45
Table 7.2	Frameworks, libraries and APIs	45
Table 7.3	Database and storage	46
Table 7.4	Hardware, cloud and deployment environment	46
Table 7.5	Development tools	47
Table 8.1	API and external service integration	53
Table 9.1	Unit / module testing	56
Table 9.2	Functional testing (platform QA suite)	57
Table 9.3	Integration testing	58
Table 9.4	System testing (AI generator suite and end-to-end scenarios)	59
Table 9.5	Performance testing	60
Table 9.6	User acceptance testing	60
Table 9.7	Experimental validation of AI generation	61
Table 9.8	Defects identified and resolved	61
Table 10.1	Objective-wise achievement	64
Table 10.2	Performance metrics	64

Table No.	Title	Page
Table 10.3	Comparative results with existing approaches	65
Table 11.1	Major challenges	68
Table 11.2	Improvements made during development	69
Table 12.1	Individual contribution	71
Table 13.1	Current limitations	73
Table 14.1	Final project status	75
Table A.1	Internship evidence checklist	85
Table A.2	Source code, repository and deployment evidence	85
Table A.3	Complete REST API endpoint catalogue	86
Table A.4	Mentor / evaluator remarks	90
Table B.1	Final hard-bound submission checklist	91

LIST OF ABBREVIATIONS / ACRONYMS

Abbreviation	Full Form / Meaning
AI	Artificial Intelligence
API	Application Programming Interface
APA	American Psychological Association (citation style)
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
DOCX	Microsoft Word Open XML Document
ER	Entity–Relationship
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple Choice Question
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORM	Object–Relational Mapping
PDF	Portable Document Format
QA	Quality Assurance
REST	Representational State Transfer
RTL	Right-to-Left (text direction)
SaaS	Software as a Service
SDLC	Software Development Life Cycle
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SSR	Server-Side Rendering
UAT	User Acceptance Testing
UDP	University Defined Project
UI / UX	User Interface / User Experience

Abbreviation	Full Form / Meaning
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XLSX	Microsoft Excel Open XML Spreadsheet

CHAPTER 1

INTRODUCTION

This chapter consolidates the initial understanding from Reporting 1 and the final project context.

1.1 Background of the Project

Digital forms are the entry point of almost every data-driven process. Colleges collect admissions and feedback, hospitals collect patient intake information, companies run employee surveys and lead-capture campaigns, and trainers conduct quizzes and assessments — all through web forms. Form-building tools such as Google Forms, Microsoft Forms, Typeform and Jotform have made it easy to host a form online, but the **authoring** step has hardly changed in a decade: a person still has to think of every question, add each field one by one, pick an input type, type the options, decide which fields are mandatory, add validations and finally style the form.

At the same time, Generative Artificial Intelligence has matured rapidly. Large Language Models (LLMs) such as Google Gemini can understand an instruction written in plain English and return structured JSON that follows a supplied schema. Multimodal models can also read PDFs and images. This creates the opportunity to describe a form in one sentence — "Create a patient intake form for a dental clinic with contact details, medical history and appointment date" — and receive a complete, correctly typed, validated and themed form within seconds.

PromptForm AI was developed during the internship to exploit this opportunity. It is a full-stack SaaS web application in which users can generate forms from prompts or uploaded documents, refine them in a visual builder, publish them through a unique link, collect responses, run graded quizzes with anti-cheat controls, analyse results on dashboards and collaborate with team members. The project domain therefore spans full-stack web engineering, applied generative AI, database design and cloud deployment.

1.2 Problem Statement

The finalized problem statement is: **"Creating high-quality online forms is a slow, manual and error-prone activity. Authors must design every field, choose input**

types and validations by hand, and re-type existing paper or PDF forms, while separate tools are needed for quizzes, analytics and collaboration. There is a need for a single platform that converts a natural-language description or an existing document into a complete, validated, shareable form in seconds, and supports the full life cycle of responses, analysis and teamwork."

In Reporting 1 the problem was initially framed only as "generate a form from a prompt". During requirement analysis and mentor reviews it evolved in three ways: (i) document digitisation (PDF, DOCX, XLSX and images) was added because many organisations already hold forms on paper; (ii) assessment use-cases required a quiz engine with scoring, timers and tab-switch detection; and (iii) the product had to be multi-user, so team workspaces, roles, invitations and activity logs were included. The problem therefore grew from a single AI feature into a complete form-management platform with AI at its core.

1.3 Need / Motivation

The practical need arises from the time cost and quality issues of manual form creation. In an informal timing exercise performed during Reporting 1, building a 10-field event registration form in a conventional builder took between 15 and 25 minutes, including choosing types for email, phone and date fields, adding dropdown options and making fields required. Mistakes such as using a plain text box for an email address, or forgetting to make the phone field mandatory, were common and only discovered after responses were collected.

There is also a technical motivation. LLM output is non-deterministic and may be malformed, so a production system cannot rely on it blindly. The project therefore explores a hybrid design in which deterministic, domain-aware rules plan and review the form while the LLM supplies natural wording and domain knowledge. This makes generation reliable, fast and usable even when the external AI service is slow or unavailable. Finally, small institutions and start-ups need an affordable all-in-one tool rather than separate subscriptions for forms, quizzes and analytics.

1.4 Objectives

The following clear and measurable objectives were defined and tracked through the four progress reports:

1. Generate a complete, editable form (title, description, 5–15 typed questions, options, validations and theme) from a natural-language prompt in under 10 seconds on average.
2. Digitise existing forms by extracting fields from uploaded PDF, DOCX, XLSX and image files.
3. Support at least 10 field types (short text, long text, MCQ, checkbox, dropdown, rating, date, time, file upload, signature, payment) together with email, phone, name and number variants with automatic validation.
4. Provide a visual drag-and-drop builder for editing questions, logic, theme, fonts and display mode (full page, wizard, chat).
5. Publish forms instantly with a unique short share link and accept public, rate-limited submissions.
6. Provide graded quizzes with points, negative marking, timers, shuffling and tab-switch (anti-cheat) detection.
7. Provide per-form and workspace analytics (views, submissions, completion rate, device and country split, sentiment) and CSV/XLSX export.
8. Enable team collaboration with workspaces, role-based access (admin / editor / viewer), e-mail invitations and activity logs.
9. Secure the platform with JWT authentication, single active session, bcrypt hashing, Helmet headers, CORS whitelist and rate limiting.
10. Validate the system with at least 40 automated test cases and deploy it on cloud infrastructure.

1.5 Scope of the Project

The project scope includes the design, implementation, testing and deployment of a responsive web application (frontend and backend), the AI generation engine, the relational database, the public form responder, analytics dashboards, team collaboration, subscription plans with AI credits and coupons, e-mail notifications and an admin panel. The deliverables are the source code in a TypeScript monorepo, the deployed application, automated test scripts, QA reports and this final report.

Native mobile applications, real payment-gateway settlement, offline form filling, on-premise LLM hosting and accessibility certification are out of scope for the current version and are discussed under future scope in Chapter 13. The scope matrix in Table 1.2 summarises the boundaries and constraints.

1.6 Expected Outcomes vs Final Outcomes

Table 1.1: *Expected outcomes compared with final outcomes*

Expected Outcome (Reporting 1)	Final Outcome (Reporting 4)	Status
Prompt-to-form generation using an LLM	Hybrid AIFormBrain + Gemini with 5-model fallback chain and JSON schema	Exceeded
Basic field types (text, MCQ, checkbox)	11 core types plus email, phone, name, number, address, country variants	Exceeded
Simple form editor	Visual builder with undo/redo, theme editor, fonts, radius, display modes	Achieved
Shareable link	Unique short share ID, public responder with full / wizard / chat modes	Achieved
Response list	Responses table, print view, CSV/XLSX export, AI response analysis	Exceeded
Quiz support (planned as optional)	Quiz engine with points, negative marks, timer, shuffle, anti-cheat	Achieved
Document upload (not planned)	PDF, DOCX, XLSX and image digitisation	Added
Single-user application	Teams, roles, invites, collaborators, activity log, notifications	Exceeded
Local deployment	Cloud deployment on Vercel + Render + managed PostgreSQL	Achieved

1.7 Report Organization

The report is organised into fifteen chapters and appendices as follows:

- **Chapter 1 – Introduction:** background, problem statement, motivation, objectives and scope.
- **Chapter 2 – Organization / Internship Profile:** organization, domain, role, responsibilities and work environment.
- **Chapter 3 – Literature Review:** domain background, existing tools and research, comparison and research gap.
- **Chapter 4 – Requirements Analysis:** functional, non-functional, hardware, software and data requirements.
- **Chapter 5 – Methodology and Project Planning:** Agile methodology, workflow, phases, progress consolidation and schedule.
- **Chapter 6 – System Analysis and Design:** existing vs proposed system, architecture, UML, module and database design.
- **Chapter 7 – Technology Stack:** languages, frameworks, database, deployment platform, tools and selection rationale.

- **Chapter 8 – System Implementation:** major modules, AI algorithm, API integration, database, security and integration.
- **Chapter 9 – Testing and Validation:** strategy, unit, functional, integration, system, performance and acceptance testing, defects.
- **Chapter 10 – Results and Performance Analysis:** results, objective achievement, metrics, comparison and screenshots.
- **Chapters 11–14:** challenges, individual contribution, limitations and future scope, and conclusion.
- **Chapter 15 – References** followed by **Appendices** with progress report summaries, screenshots and repository evidence.

Table 1.2: *Scope Matrix*

In Scope	Out of Scope	Constraints
AI form generation from prompt and documents	Native Android / iOS apps	Gemini API quota and latency
Visual builder, themes, display modes	Real payment-gateway settlement	Free-tier cloud hosting (cold starts)
Public responder and submissions	Offline form filling	Internship duration of one semester
Quiz engine with anti-cheat	Self-hosted / fine-tuned LLM	Single developer with mentor reviews
Analytics, export, sentiment	WCAG accessibility certification	Browser support: modern evergreen browsers
Teams, roles, invites, notifications	Enterprise SSO (SAML)	E-mail delivery limits of SMTP provider
Plans, credits, coupons, admin panel	Multi-region data residency	PostgreSQL as the only data store

CHAPTER 2

ORGANIZATION / INTERNSHIP / TRAINING PROFILE

2.1 Organization / Company Overview

The internship was carried out at **[COMPANY / ORGANIZATION NAME]**, a software development organisation that builds web and cloud applications for clients in education, healthcare, retail and services. The organisation follows modern engineering practices — Git-based version control, code reviews, continuous deployment to cloud platforms and Agile sprint planning — and increasingly integrates generative-AI capabilities into its products.

The organisation provided the problem context for PromptForm AI: many of its clients repeatedly request custom registration, feedback, quiz and intake forms, and building each form manually consumes developer and client time. An internal product that automates form creation and analysis was therefore identified as a high-value project suitable for an internship of one semester.

2.2 Department / Domain

The work was carried out in the **Product Engineering / Web Development** team, within the domain of full-stack web development and applied generative AI. The team is responsible for designing user-facing web applications, REST APIs, databases and their cloud deployment, and for evaluating new AI services for use in client products.

The working environment combined daily stand-up updates, weekly mentor reviews and fortnightly demonstrations. Tasks were tracked as user stories, code was maintained in a Git repository and every significant feature was demonstrated to the company mentor before being merged into the main branch.

2.3 Role and Responsibilities

The intern worked as a **Full-Stack Developer (AI Integration)** and was responsible for the complete life cycle of the project:

- Analysing requirements with the mentor and converting them into user stories and functional / non-functional requirements.

- Designing the system architecture, database schema (Prisma models) and REST API contract.
- Implementing the Next.js frontend: landing page, authentication, dashboard, AI generator, visual builder, templates, analytics, team, billing and public responder pages.
- Implementing the Express backend: routes, middlewares, services, the AIFormBrain engine and the Gemini integration.
- Implementing security controls — JWT, refresh tokens, single active session, bcrypt, Helmet, CORS and rate limits.
- Writing automated QA and AI-generation test scripts, logging defects and verifying fixes.
- Deploying the application to Vercel and Render, configuring environment variables and the managed database.
- Preparing progress reports, demonstrations and this final report.

2.4 Development / Work Environment

Development was carried out on a Windows 10 laptop using Visual Studio Code with TypeScript, ESLint and Prettier. Node.js 20+ and npm workspaces were used to manage a monorepo with three packages: frontend, backend and packages/shared. PostgreSQL ran locally (and through docker-compose together with Redis) during development, while production used a managed PostgreSQL instance. Git was used for version control with descriptive commits; the repository contains 66 commits at the time of writing. Postman and custom Node.js scripts were used for API testing, and the browser developer tools were used for UI debugging.

2.5 Internship Details

Table 2.1: *Internship details*

Particular	Details
Organization	[COMPANY / ORGANIZATION NAME]
Department / Team	Product Engineering – Web & AI
Role	Full-Stack Developer Intern (AI Integration)
Duration	[START DATE] to [END DATE]
Mode	[On-site / Hybrid / Remote]
Company Mentor	[COMPANY MENTOR NAME]
Institute Mentor	[INSTITUTE MENTOR NAME]

Particular	Details
Project	PromptForm AI – AI-Powered Intelligent Form Generation and Analytics Platform
Primary Technologies	TypeScript, Next.js 15, React 19, Express 4, Prisma 5, PostgreSQL, Google Gemini API
Deliverables	Source code, deployed application, test reports, progress reports, final report

CHAPTER 3

LITERATURE REVIEW / BACKGROUND STUDY

This chapter consolidates the literature and research work recorded during the progress reports, and discusses its relevance instead of only listing sources.

3.1 Domain Background

Online form builders. Web forms are defined in HTML through input elements and validated on both client and server. Hosted builders abstract this into a graphical editor where each question is a component with a type, label, options and validation rules. Responses are stored in a database and presented as tables or charts. Usability research on web forms recommends clear labels, appropriate input types, inline validation and minimal required fields to reduce abandonment (Wroblewski, 2008).

Large Language Models. Transformer-based models (Vaswani et al., 2017) trained on large corpora can follow instructions and generate text. Instruction-tuned models such as Google Gemini accept a system instruction, a user prompt and optional files (images, PDFs) and can be constrained to emit JSON that conforms to a response schema. This makes them suitable for producing structured objects such as a form definition. However, LLMs may hallucinate, produce inconsistent types or exceed latency budgets, so production systems combine them with validation and fallbacks (Brown et al., 2020; Google, 2024).

Document understanding. Extracting fields from existing forms requires text extraction from PDFs (pdf-parse), Word documents (mammoth) and spreadsheets (SheetJS), and multimodal understanding for scanned images. Multimodal LLMs can read the image directly, avoiding a separate OCR pipeline.

Modern web architecture. React-based frameworks such as Next.js provide routing, server-side rendering and optimised builds, while Node.js/Express is a lightweight choice for JSON REST APIs. Object–relational mappers such as Prisma provide type-safe database access and migrations over PostgreSQL. Security guidance from OWASP recommends hashed passwords, short-lived tokens, secure headers, CORS restrictions and rate limiting for such APIs.

3.2 Existing Approaches / Related Work

Table 3.1: *Review of existing approaches and related work*

Source / Work	Focus	Approach	Key Finding	Relevance
Google Forms	General forms and quizzes	Manual GUI builder, Sheets export	Free and simple; no AI generation of whole forms, limited themes and analytics	Baseline for comparison and UX expectations
Typeform	Conversational forms	One-question-at-a-time UI, logic jumps	High completion rates, but manual authoring and costly plans	Inspired wizard and chat display modes
Jotform	Enterprise forms, PDFs	Templates, widgets, PDF editor	Large template library; templates still need manual editing	Inspired template marketplace
Microsoft Forms	Education / Office 365	Manual builder, basic quizzes	Good quiz grading; tied to Microsoft ecosystem	Quiz scoring requirements
Vaswani et al. (2017)	Transformer architecture	Self-attention networks	Foundation of modern LLMs	Theoretical basis of the AI engine
Brown et al. (2020)	Few-shot LLMs (GPT-3)	Prompting without fine-tuning	LLMs perform new tasks from instructions/examples	Justifies prompt-based generation
Google Gemini API docs (2024)	Structured output	responseMimeType + responseSchema	Model can be forced to emit schema-valid JSON	Used directly for form JSON
Wroblewski (2008)	Web form design	Empirical UX guidelines	Correct input types and labels reduce errors	Rules in field intelligence engine
OWASP Top 10 (2021)	Web security	Risk catalogue	Broken access control and auth failures are top risks	Security design of the API

3.3 Comparative Analysis

Table 3.2: *Comparative analysis of existing approaches and the proposed project*

Parameter	Existing Approach	Proposed Project	Observation
Form creation	Manual, field by field	Prompt or document → full form in seconds	Major time saving
Field type selection	User chooses	Field intelligence engine infers type and validation	Fewer data-quality errors
Existing paper/PDF forms	Retyped manually	Uploaded and digitised	New capability
Quiz / assessment	Basic scoring	Points, negative marks, timer, shuffle, anti-cheat	Suitable for exams

Parameter	Existing Approach	Proposed Project	Observation
Display modes	Usually one layout	Full page, wizard, chat	Better respondent UX
Analytics	Summary charts	Views, completion, devices, countries, sentiment, AI analysis	Richer insight
Collaboration	Share edit link	Teams, roles, invites, activity log, notifications	Controlled access
Reliability of AI	Not applicable	Rule engine + 5-model LLM fallback	Works under AI failure
Cost	Subscription per tool	Free tier + single Pro plan	Affordable for small users

3.4 Research Gap / Project Justification

The study shows that popular builders have matured in hosting and response collection but leave the core authoring task to the user. Some products have recently added AI "question suggestions", yet these typically generate text only and still require manual selection of field types, validation and theming. None of the reviewed tools combines (a) whole-form generation from a prompt, (b) digitisation of existing documents, (c) graded quizzes with anti-cheat, (d) analytics with sentiment and (e) team collaboration in a single affordable platform.

Research on LLMs also highlights the gap between impressive demonstrations and reliable production behaviour: outputs must be structured, validated and available within a strict latency budget. PromptForm AI addresses this gap with a hybrid architecture in which a deterministic AIFormBrain plans, validates and reviews every form, and the LLM is used through schema-constrained JSON with a multi-model fallback chain. This justifies the project both as a useful product and as a practical study of dependable LLM integration.

CHAPTER 4

REQUIREMENTS ANALYSIS AND PROJECT SCOPE

Requirements were gathered through discussions with the company mentor, study of existing form tools (Chapter 3) and analysis of typical client form requests. They were prioritised using the MoSCoW technique and traced to implementation and test cases.

4.1 Functional Requirements

Table 4.1: *Functional requirements*

Req. ID	Requirement	Priority	Source / Basis	Status
FR-01	User registration, login, logout, password reset and Google OAuth sign-in	Must	Mentor / SaaS norm	Done
FR-02	Generate a complete form from a natural-language prompt	Must	Core problem	Done
FR-03	Generate a form from an uploaded PDF, DOCX, XLSX or image	Should	Client feedback	Done
FR-04	Edit questions, types, options, required flag, order and logic in a visual builder	Must	Core problem	Done
FR-05	Customise theme colour, font, corner radius and display mode	Should	UX study	Done
FR-06	Publish a form with a unique short share link; close or reopen it	Must	Core problem	Done
FR-07	Accept public submissions with validation and response limits	Must	Core problem	Done
FR-08	Create graded quizzes with points, negative marks, timer, shuffle and anti-cheat	Should	Education clients	Done
FR-09	View responses, print them and export CSV / XLSX	Must	Mentor	Done
FR-10	Per-form and workspace analytics with device, country and sentiment insights	Should	Mentor	Done
FR-11	Template marketplace with live preview and “use template”	Could	Competitor study	Done
FR-12	Teams with admin/editor/viewer roles, e-mail invitations and activity log	Should	Client feedback	Done
FR-13	Form-level collaborators (editor / viewer)	Could	Mentor	Done
FR-14	Free and Pro plans, AI credits, coupons and transaction receipts	Should	Business model	Done
FR-15	Live in-app notifications and e-mail alerts for new responses	Could	UX study	Done
FR-16	AI assistant chat to modify a form conversationally	Could	Innovation	Done

Req. ID	Requirement	Priority	Source / Basis	Status
FR-17	Admin panel to monitor users, coupons and platform metrics	Could	Mentor	Done

4.2 Non-Functional Requirements

Table 4.2: *Non-functional requirements*

Req. ID	Category	Requirement	Target / Measure	Status
NFR-01	Performance	AI generation response time	< 10 s average, 45 s hard timeout	Met
NFR-02	Performance	Standard API response time	< 500 ms for CRUD on local DB	Met
NFR-03	Security	Password storage	bcrypt hash, min. 10 characters	Met
NFR-04	Security	Authentication	JWT access + refresh, single active session	Met
NFR-05	Security	Abuse protection	Rate limits: API 1000/15 min, AI 60/min, submit 60/min (prod)	Met
NFR-06	Security	HTTP hardening	Helmet headers, CORS whitelist, 20 MB body limit	Met
NFR-07	Reliability	AI availability	Fallback across 5 Gemini models + rule engine	Met
NFR-08	Usability	Responsive UI	Usable from 360 px mobile to desktop; light/dark theme	Met
NFR-09	Maintainability	Type safety and structure	TypeScript everywhere, layered routes/services	Met
NFR-10	Scalability	Stateless API	JWT-based, horizontal scaling, indexed tables	Met
NFR-11	Observability	Logging and health	Request logger, /health liveness & readiness	Met
NFR-12	Portability	Deployment	Runs locally, in Docker and on Vercel/Render	Met

4.3 Hardware Requirements (if applicable)

Table 4.3: *Hardware requirements*

Component	Specification	Purpose	Availability
Development machine	Intel Core i5 / Ryzen 5, 8–16 GB RAM, SSD	Coding, local servers, builds	Available
Storage	≥ 10 GB free (node_modules, DB, builds)	Dependencies and database	Available
Network	Broadband ≥ 10 Mbps	Gemini API, package install, deployment	Available
Server (production)	Render web service, 512 MB+ RAM	Express backend	Cloud
Database (production)	Managed PostgreSQL instance	Persistent data	Cloud
End-user device	Any modern browser on desktop / mobile	Using the application	User side

4.4 Software Requirements

Table 4.4: *Software requirements*

Software / Tool	Version	Purpose	Source / License
Node.js	20.x / 24.x	JavaScript runtime for backend and tooling	MIT
TypeScript	5.4+	Typed language for all packages	Apache-2.0
Next.js	15.x	React framework for the frontend	MIT
React / React DOM	19 RC	UI library	MIT
Express	4.19	REST API framework	MIT
Prisma ORM	5.12	Database schema, migrations and client	Apache-2.0
PostgreSQL	15/16	Relational database	PostgreSQL License
Redis (optional)	7.x	Cache / BullMQ queue	BSD
Google Gemini API	v1beta	LLM generation and document understanding	Commercial API
Visual Studio Code	Latest	IDE	MIT (Code-OSS)
Git / GitHub	2.x	Version control and hosting	GPL-2.0 / SaaS
Docker / docker-compose	24+	Local PostgreSQL and Redis	Apache-2.0
Postman	Latest	API testing	Freeware
Vercel / Render	Cloud	Frontend and backend hosting	SaaS

4.5 Data / Dataset Requirements (if applicable)

PromptForm AI does not train its own machine-learning model, so no training dataset was required. The system relies on application data created by users and on curated domain knowledge embedded in the AIFormBrain rules. The data used during development and testing is summarised below.

Table 4.5: *Data requirements*

Data	Source	Type	Size / Records	Pre-processing	Purpose
Seed users and templates	prisma/seed.ts	Relational	2 users, starter templates	Hashed passwords	Local development
Test prompts	QA scripts	Text	11 AI scenarios	None	AI generation tests
Sample documents	Prepared PDF / image forms	PDF, PNG	Several files	Text extraction / base64	Digitisation tests
Domain keyword lists	AIFormBrain modules	Rules	Healthcare, education, HR, events, surveys…	Normalisation	Domain detection
QA user accounts	Test runner	Relational	~58 users in local DB	Generated per run	Auth / limits tests
Responses	Demo submissions	JSON	16 demo responses	Zod validation	Analytics screenshots

4.6 Constraints and Assumptions

- **API dependency:** generation quality and latency depend on the Google Gemini API; quotas and occasional outages were assumed and handled with fallbacks.
- **Hosting:** free-tier cloud hosting introduces cold-start delays of several seconds on the first request after inactivity.
- **Time:** the project had to be completed within one internship semester with four progress reporting milestones.
- **Team:** a single developer implemented the system with weekly mentor reviews, so features were prioritised using MoSCoW.
- **Assumption:** users have a modern browser with JavaScript enabled and a stable Internet connection.
- **Assumption:** uploaded documents are legible and contain form-like content; heavily handwritten scans may produce partial results.

- **Payments:** the billing flow records plans, coupons and receipts but does not settle real money through a payment gateway.

CHAPTER 5

METHODOLOGY AND PROJECT PLANNING

5.1 Final Development / Research Methodology

The project followed an **Agile, iterative and incremental** methodology based on Scrum practices adapted for a single developer. Work was divided into two-week sprints. Each sprint began with planning (selecting user stories from the prioritised backlog), continued with design, implementation and testing, and ended with a demonstration to the company mentor and a short retrospective. Feedback from the demonstrations was added to the backlog, which is why features such as document digitisation and team collaboration were introduced after the first progress report.

Agile was preferred over the Waterfall model because the AI component involved uncertainty: prompt designs, model choices and the balance between rule-based and LLM generation could only be decided through experimentation. Short iterations allowed quick evaluation and course correction. The four university progress reports acted as major milestones (release reviews) on top of the sprint cycle.

For the AI component an experimental approach was used: candidate prompts and system instructions were evaluated against a fixed set of test prompts (medical intake, quiz, survey, event registration, job application). Outputs were checked for schema validity, correct field types, sensible option lists and latency, and the best configuration was retained. This led to the final hybrid design described in Chapter 8.

5.2 Project Workflow

The overall workflow of the project, from problem study to evaluation, is shown in Figure 5.1. Feedback from each reporting milestone flowed back into requirements and design.

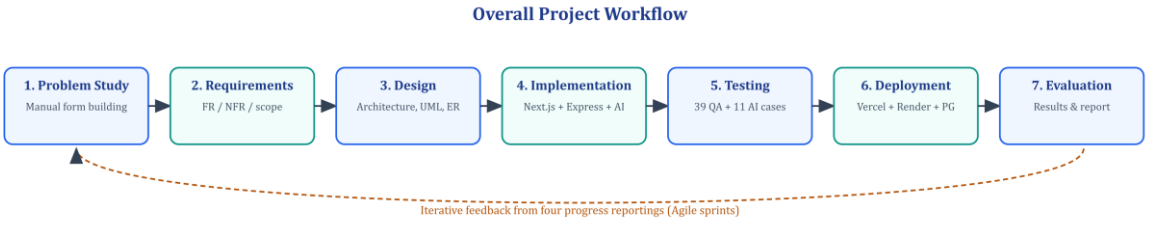


Figure 5.1: *Overall Project Workflow*

5.3 Project Phases and Deliverables

Table 5.1: *Project phases and deliverables*

Phase	Activities	Deliverables	Evidence / Status
1. Problem study	Study existing form tools, interview mentor, define problem and objectives	Problem statement, objectives	Reporting 1 – Completed
2. Requirements	FR/NFR, MoSCoW prioritisation, feasibility	Requirement specification	Chapter 4 – Completed
3. Design	Architecture, UML, ER model, API contract, UI wireframes	Design documents, Prisma schema	Reporting 2 – Completed
4. Core implementation	Auth, forms CRUD, builder, public responder	Working MVP	Reporting 2 – Completed
5. AI implementation	AIFormBrain, Gemini integration, document parsing, quiz engine	AI generation feature	Reporting 3 – Completed
6. Advanced features	Analytics, teams, notifications, billing, admin	Feature-complete product	Reporting 3 – Completed
7. Testing	QA suite, AI test suite, bug fixing, re-verification	QA reports (50 tests)	Reporting 3/4 – Completed
8. Deployment	Vercel, Render, managed PostgreSQL, env configuration	Live application	Reporting 4 – Completed
9. Documentation	Progress reports, final report, viva preparation	Final hard-bound report	Reporting 4 – Completed

5.4 Progress Report Consolidation

Table 5.2: *Progress report consolidation*

Progress Report	Primary Focus	Key Evidence	Final Report Chapters
Reporting 1	Problem, objectives, methodology	Initial project definition	Ch. 1, Ch. 5, Ch. 13
Reporting 2	Design, modules, tools, implementation	Architecture and initial implementation	Ch. 4, Ch. 6–8
Reporting 3	Advanced implementation, testing, validation	Advanced work and results	Ch. 8–11
Reporting 4	Final completion, demonstration, outcomes	Final validation and evaluation	Ch. 9–14

5.5 Project Schedule / Timeline

The planned and actual schedule (weeks relative to the internship start date) is given below and visualised as a Gantt chart in Figure 5.2. Minor slippage in the AI engine phase was absorbed by overlapping the testing phase with feature completion.

Table 5.3: *Project schedule – planned vs actual*

Task / Phase	Planned Start	Planned End	Actual Start	Actual End	Status
Problem study & literature	W1	W2	W1	W2	Completed
Requirement analysis	W2	W3	W2	W3	Completed
Architecture & UML design	W3	W4	W3	W4	Completed
Database schema (Prisma)	W4	W5	W4	W5	Completed
Auth & core APIs	W5	W7	W5	W7	Completed
AI engine (AIFormBrain + Gemini)	W6	W9	W6	W10	Completed (+1 week)
Builder, templates, public forms	W7	W10	W7	W10	Completed
Analytics, teams, notifications	W9	W12	W9	W12	Completed
Testing & bug fixing	W10	W13	W10	W13	Completed
Deployment (Vercel / Render)	W12	W13	W12	W13	Completed
Documentation & final report	W13	W14	W13	W14	Completed

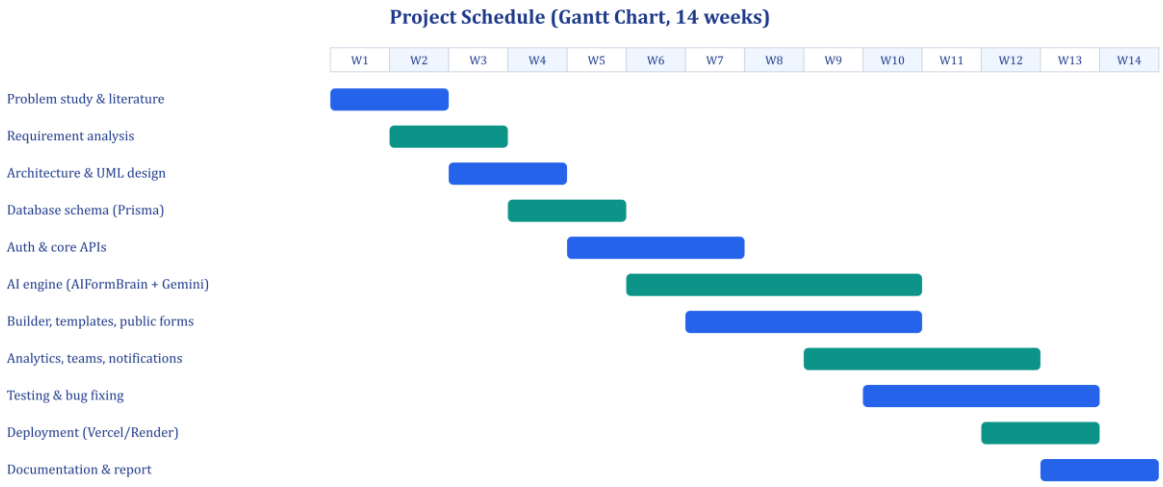


Figure 5.2: *Project Schedule (Gantt Chart)*

CHAPTER 6

SYSTEM ANALYSIS AND DESIGN

6.1 Existing System / Current Practice

In current practice a form author opens a hosted builder, creates an empty form and adds questions one at a time. For every question the author types a label, selects a type from a menu, types each option for choice questions, toggles the required flag and optionally adds validation such as an e-mail pattern. Themes are chosen from a small set of presets. If the organisation already has the form on paper or in a PDF, every field is re-typed. For assessments, correct answers and marks are entered manually for each question.

After publication, responses are viewed as a list or exported to a spreadsheet; deeper analysis such as completion rate, drop-off, device split or sentiment of text answers is either unavailable or requires external tools. Collaboration is usually limited to sharing an edit link, which gives full control to anyone holding it. The main weaknesses of the existing practice are therefore: high authoring time, inconsistent field types and validation, no digitisation of existing documents, limited quiz integrity controls, shallow analytics and coarse access control.

6.2 Proposed System

The proposed system, PromptForm AI, replaces manual authoring with AI generation while keeping the author in control. The author types a description or uploads a document; the backend validates the request, checks plan limits and AI credits, extracts text from documents, and runs the AIFormBrain pipeline, which detects intent, language, domain and industry, plans the fields with appropriate types and validations, and calls Gemini for natural wording. The quality reviewer corrects the result, which is saved as a published form with a unique share link and opened in the visual builder for refinement.

Respondents fill the form in full-page, wizard or chat mode. Submissions are validated on the server (required fields, password protection, response limits, anti-cheat flags) and stored together with browser metadata. Analytics events (views, starts, submissions, drop-outs) feed per-form and workspace dashboards, and the owner can export responses, ask the AI to analyse them, and receive notifications. Teams, roles and

collaborators provide controlled access. The key advantages are speed, consistency of field types, digitisation, integrity of quizzes, richer analytics and secure collaboration.

6.3 System Architecture

PromptForm AI follows a three-tier client–server architecture (Figure 6.1). The presentation tier is a Next.js single-page experience; the application tier is an Express REST API composed of a gateway layer (security middleware), routers, the AIFormBrain engine and services; the data tier is PostgreSQL accessed through Prisma. External services include the Gemini API and an e-mail provider. The API is stateless — identity travels in a JWT — so it can be scaled horizontally.

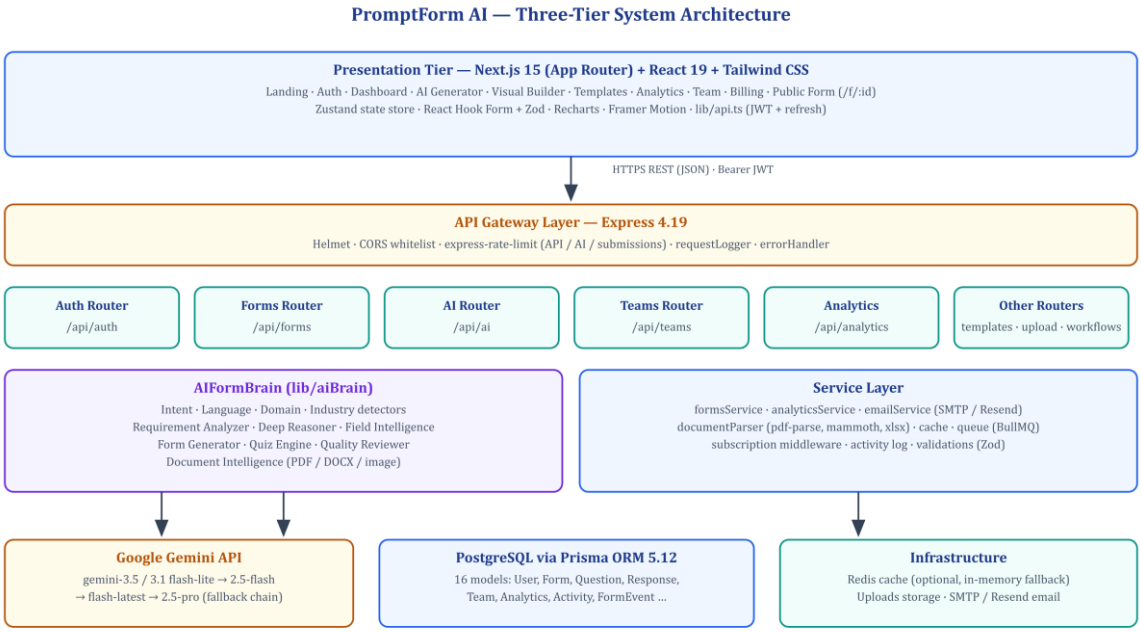


Figure 6.1: *Final System Architecture*

6.4 Workflow / Process Flow

The core process — turning a prompt or document into a published form — is shown in Figure 6.2. Each step either enriches the form definition or enforces a rule (credits, plan limits, schema validity) before the next step is executed.

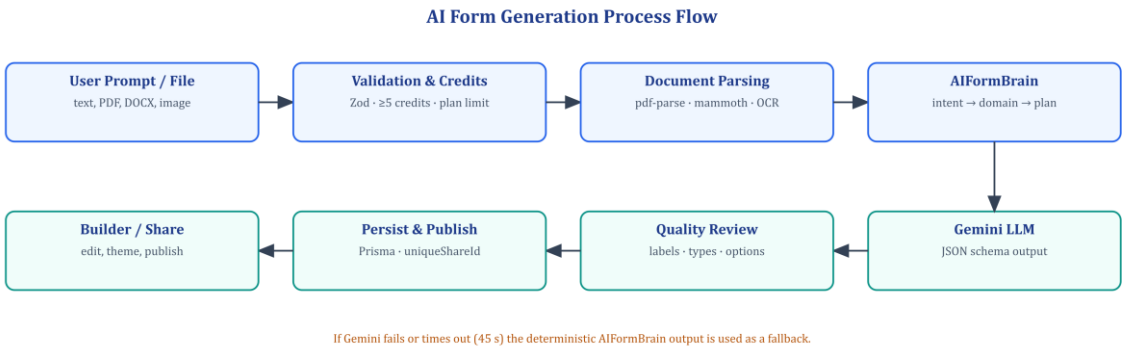


Figure 6.2: *Final Workflow / Process Flow*

6.5 UML / Design Diagrams

The design was documented using UML and structured-analysis diagrams: the use case diagram (Figure 6.3) identifies actors and their goals; the Level-1 data flow diagram (Figure 6.4) shows processes and data stores; the class diagram (Figure 6.5) shows the main engine and service classes; the sequence diagram (Figure 6.6) details AI generation; the activity diagram (Figure 6.7) describes form submission; and the ER diagram (Figure 6.8) presents the database schema.

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- ER Diagram / Database Schema
- Other domain-specific diagrams (Data Flow Diagram – Level 1)

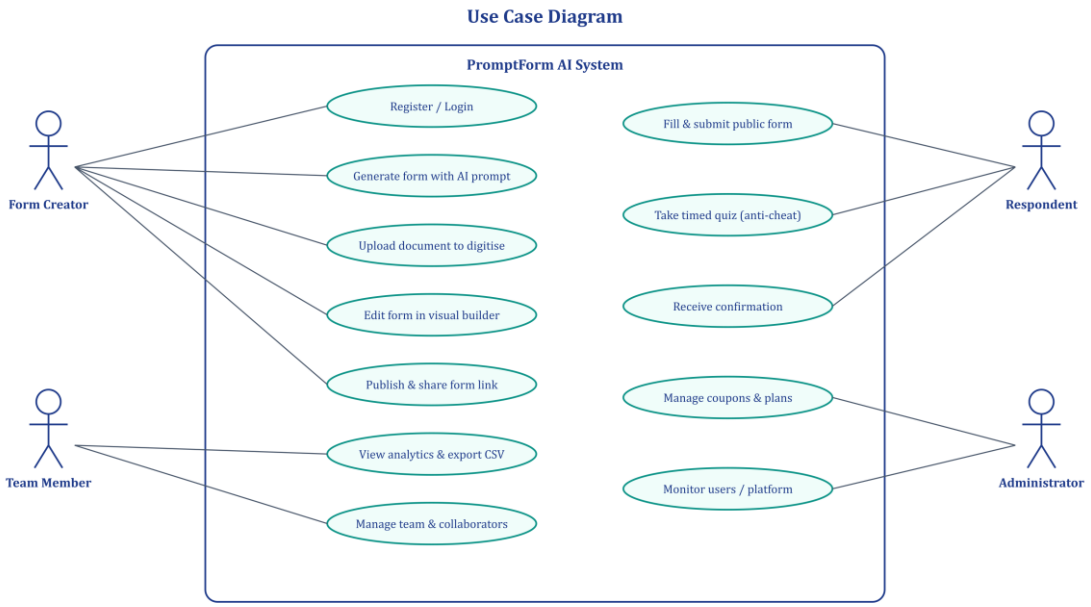


Figure 6.3: *Major UML / ER / Design Diagram — Use Case Diagram*

Four actors interact with the system. The **Form Creator** generates, edits, publishes and analyses forms; a **Team Member** collaborates on shared forms according to an assigned role; a **Respondent** fills public forms and quizzes without needing an account; and the **Administrator** manages plans, coupons and platform-level monitoring.

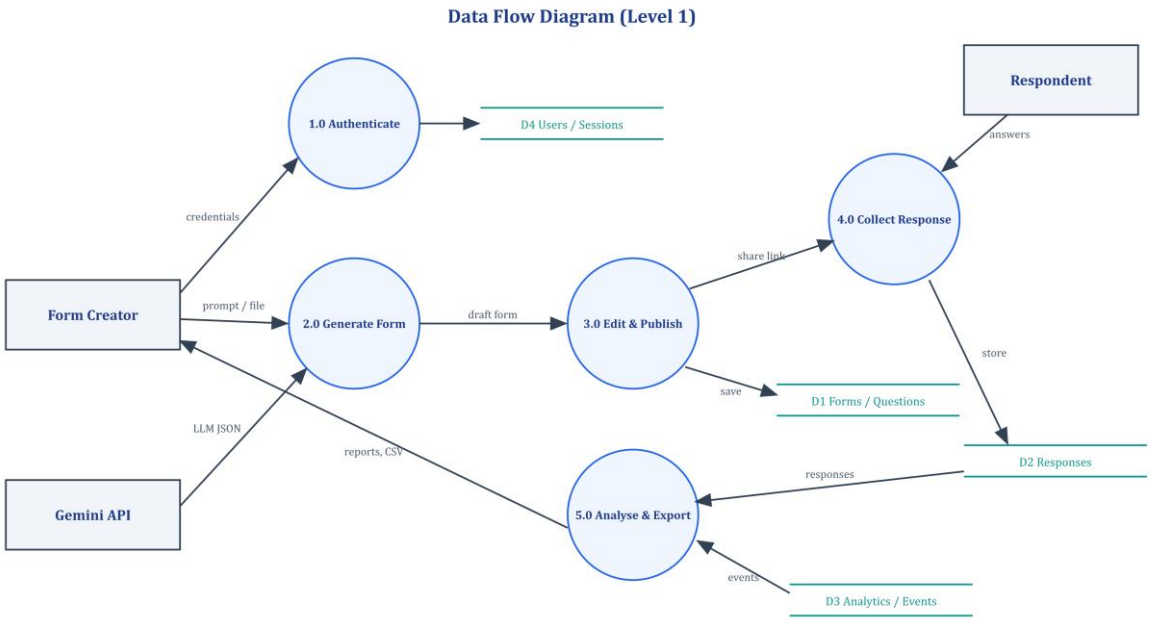


Figure 6.4: *Data Flow Diagram (Level 1)*

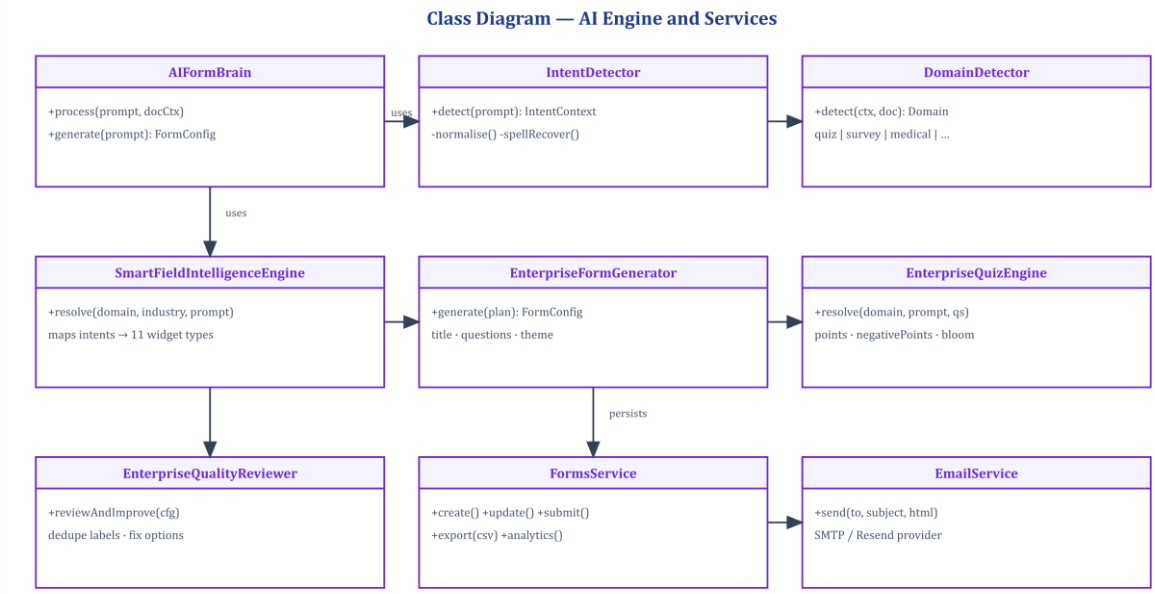


Figure 6.5: *Class Diagram — AI Engine and Services*

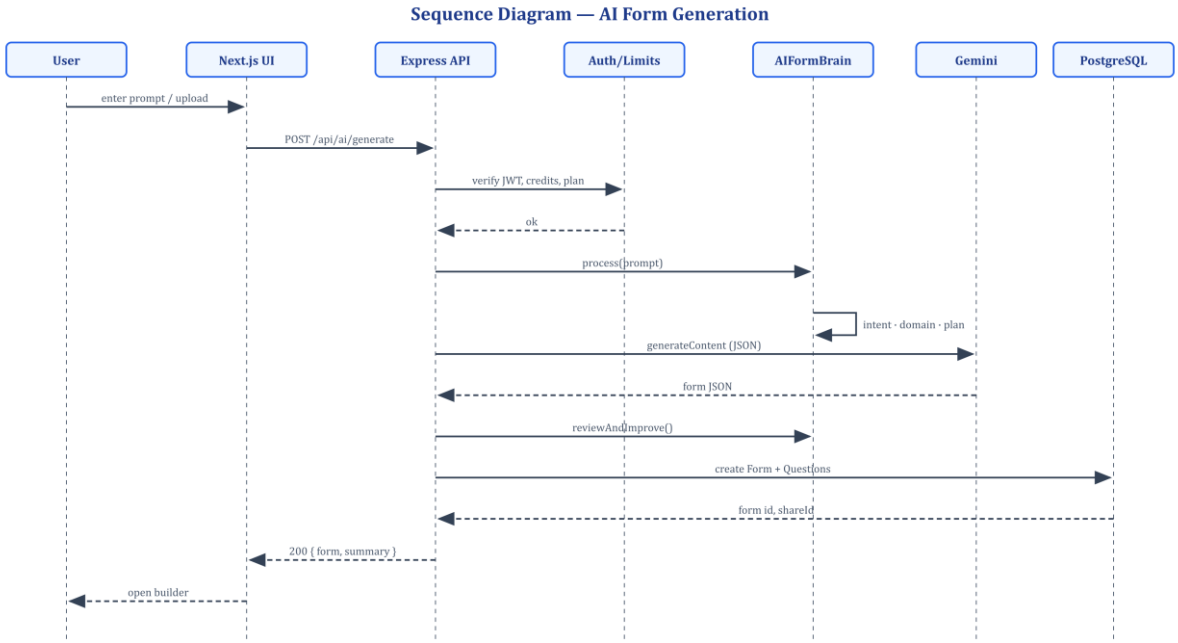


Figure 6.6: *Sequence Diagram — AI Form Generation*

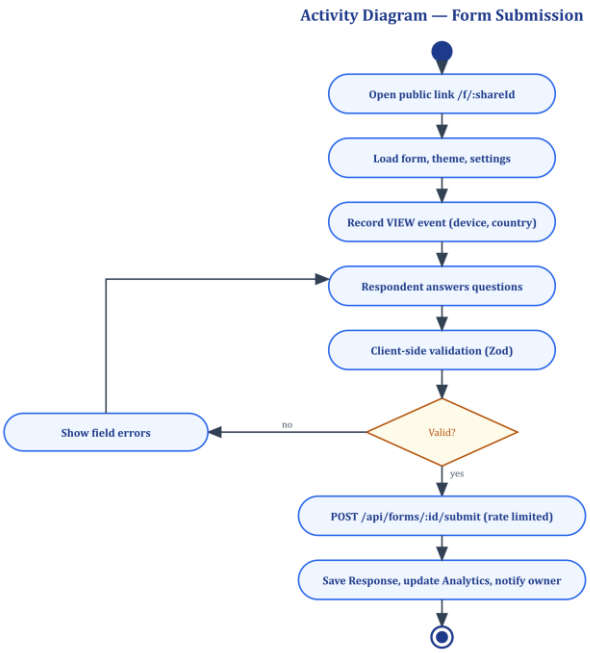


Figure 6.7: *Activity Diagram — Form Submission*

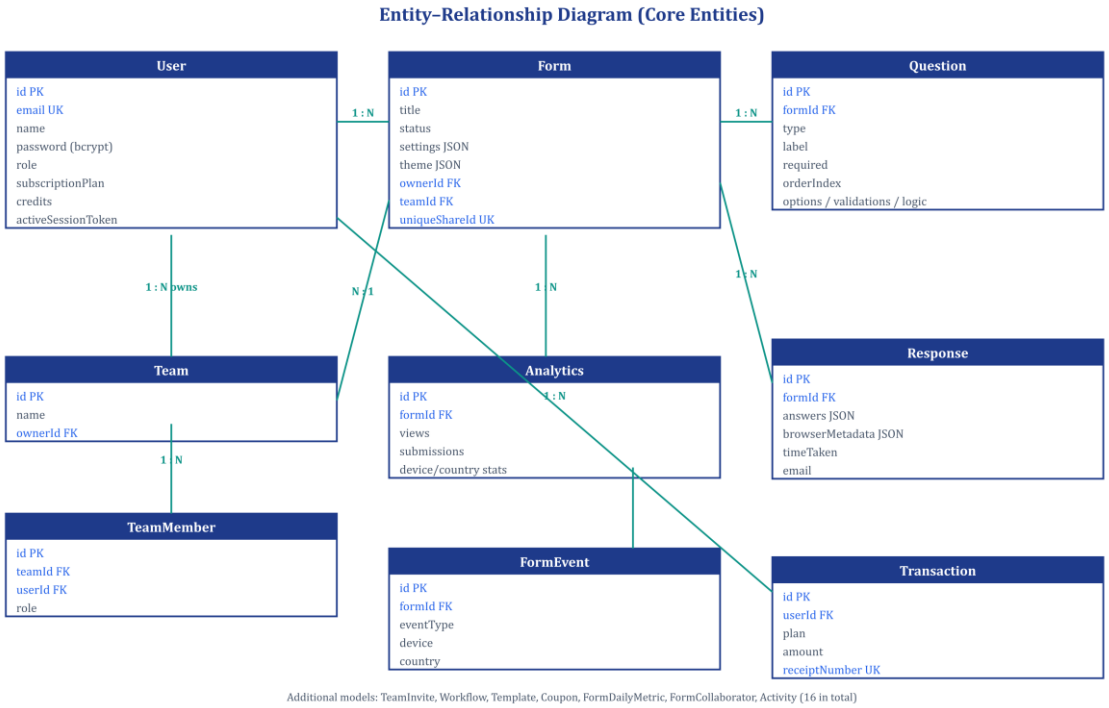


Figure 6.8: *ER Diagram / Database Schema*

6.6 Module-wise Design

Table 6.1: *Module-wise design*

Module	Purpose	Inputs	Processing	Outputs	Dependencies
Authentication	Identify users securely	Email, password, OAuth code	bcrypt verify, JWT issue, session token	Access/refresh tokens	User model, JWT

Module	Purpose	Inputs	Processing	Outputs	Dependencies
AI Generator	Create forms automatically	Prompt, file, options	Parsing, AIFormBrain, Gemini, review	Saved form + summary	Gemini, documentParser
Visual Builder	Edit forms	Form JSON, user edits	State store, undo/redo, PUT questions	Updated form	Forms API, Zustand
Templates	Start from ready forms	Template id	Clone structure into new form	New form	Template model
Public Responder	Collect answers	Share id, answers	Client + server validation, events	Response record	Forms submit API
Quiz Engine	Assessments	Graded questions, answers	Scoring, negative marks, timer, tab-switch	Score, flags, CSV	Response metadata
Analytics	Insight into performance	Events, responses	Aggregation, daily metrics, sentiment	Charts, KPIs	FormEvent, Analytics
Teams	Collaboration	Team, invites, roles	RBAC checks, invite tokens, activity log	Members, access	Team models, email
Billing	Plans and credits	Plan, coupon	Limit checks, discounts, receipts	Transaction	subscription middleware
Notifications	Inform owners	Submission, invite events	Live feed + e-mail dispatch	Alerts, e-mails	emailService

6.7 Database / Data Design

The relational schema contains 16 Prisma models. Flexible, form-specific structures (settings, theme, options, validations, logic, answers) are stored as PostgreSQL JSON columns, while identity and relationships use UUID keys, foreign keys with cascade rules, unique constraints and indexes on frequently filtered columns.

Table 6.2: *Database entities*

Entity / Table	Key Fields	Relationships	Purpose
User	id, email (unique), password, role, subscriptionPlan, credits, activeSessionToken	1:N Form, Team, Transaction, Activity	Accounts and plans
Form	id, title, status, settings, theme, ownerId, teamId, uniqueShareId	N:1 User/Team; 1:N Question, Response	Form definition
Question	id, formId, type, label, required, orderIndex, options, validations, logic	N:1 Form	Form fields

Entity / Table	Key Fields	Relationships	Purpose
Response	id, formId, answers, browserMetadata, timeTaken, email	N:1 Form	Submitted answers
Team / TeamMember	team id, name, ownerId; member role	N:M User via TeamMember	Workspaces and RBAC
TeamInvite	email, role, token (unique), status, expiresAt	N:1 Team	E-mail invitations
FormCollaborator	formId, userId, role (unique pair)	N:M Form–User	Per-form sharing
Analytics / FormDailyMetric	views, submissions, device/country stats, date	N:1 Form	Aggregated metrics
FormEvent	eventType, questionId, device, country	N:1 Form	Raw analytics events
Workflow	trigger, action, config, active	N:1 Form	Automation on submit
Template	title, category, structure	—	Template marketplace
Transaction / Coupon	plan, amount, receiptNumber; code, discountPercent	N:1 User	Billing records
Activity	action, targetTitle, details, metadata	N:1 User/Team/Form	Audit trail

6.8 Interface / UI Design (if applicable)

The interface follows a warm, minimal design language: a cream background, a brown primary accent, rounded cards, Inter typography and generous spacing, with a full dark mode through next-themes. Navigation in the authenticated area uses a top bar with Overview, Forms, AI Generator, Templates, Analytics, Team and Settings, plus search, theme toggle, notifications and profile menu. The builder uses a three-pane layout — field palette, live canvas and properties panel — so that every change is visible immediately.

The public landing page (Figure 6.9) introduces the product, demonstrates prompt-based generation and links to templates and pricing. All pages are responsive and were checked at mobile, tablet and desktop widths.

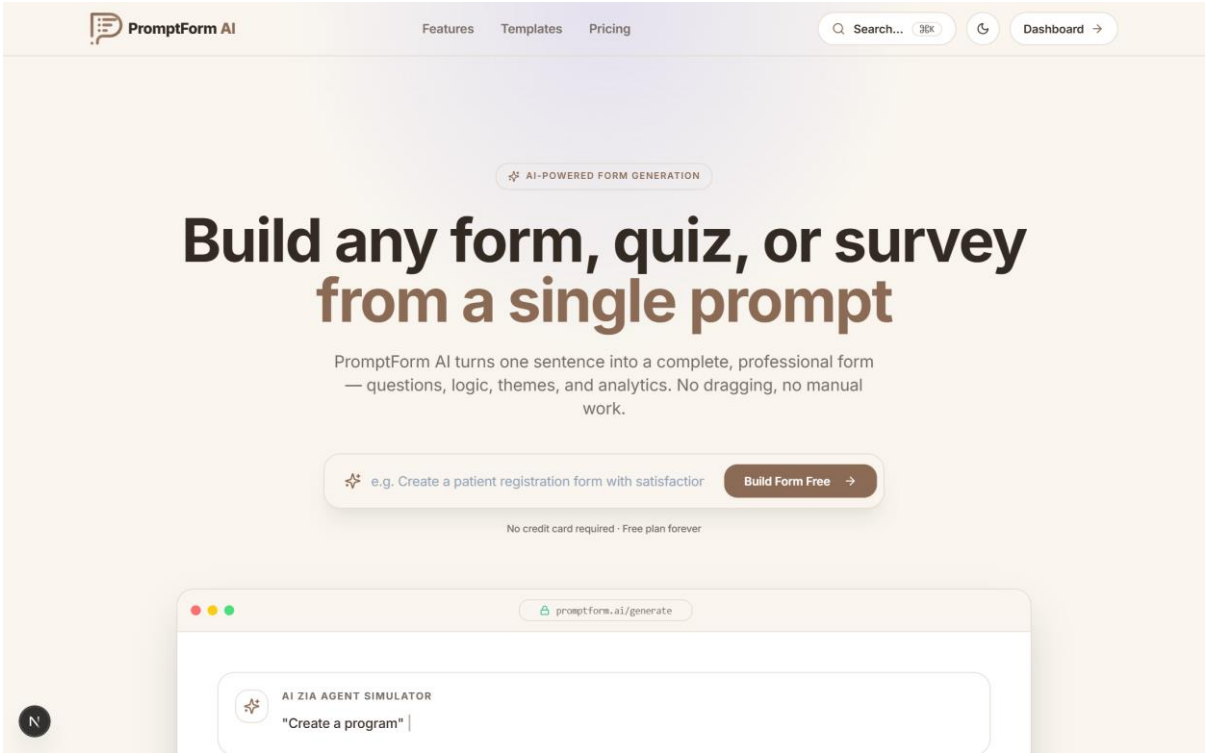


Figure 6.9: *Major Interface / UI Design*

CHAPTER 7

TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

7.1 Programming Languages

Table 7.1: *Programming languages*

Technology	Version	Purpose	Major Usage
TypeScript	5.4 / 5.5	Typed superset of JavaScript	All frontend, backend and shared code
JavaScript (ES2022)	Node 20+	Scripts and tooling	dev orchestrator, QA scripts, build helpers
SQL (PostgreSQL dialect)	15/16	Database language	Generated by Prisma migrations, JSON queries
HTML5 / CSS3	—	Markup and styling	Rendered by React, styled with Tailwind CSS
Prisma Schema Language	5.12	Data modelling	schema.prisma with 16 models

7.2 Frameworks / Libraries / APIs

Table 7.2: *Frameworks, libraries and APIs*

Tool / Framework / API	Version	Purpose	Usage
Next.js (App Router)	15.5	React framework	Routing for 29 pages, builds, dev server on port 4500
React / React DOM	19 RC	UI rendering	Components, hooks
Tailwind CSS	3.4	Utility-first CSS	Design system, dark mode
Zustand	5.0	State management	Builder and session stores
React Hook Form + Zod	7.53 / 3.23	Forms and schemas	Client validation
Recharts	3.10	Charts	Analytics dashboards
Framer Motion, Lucide	11 / 0.453	Animation, icons	UI polish
Express	4.19	HTTP server	73 REST routes
jsonwebtoken, bcryptjs	9.0 / 2.4	Auth	Tokens and password hashing
Helmet, cors, express-rate-limit	7.1 / 2.8 / 7.2	Security	Headers, origins, throttling
Multer	1.4	File upload	Documents and response files
pdf-parse, mammoth, xlsx	2.4 / 1.12 / 0.18	Document parsing	PDF, DOCX, spreadsheet text

Tool / Framework / API	Version	Purpose	Usage
Nodemailer / Resend	10.0	E-mail	Invites, notifications
BullMQ, redis	5.79 / 4.6	Queue and cache	Background jobs (optional)
geoip-lite, ua-parser-js	2.0 / 2.0	Analytics enrichment	Country and device detection
Google Gemini API	v1beta	LLM	Form generation, document reading, analysis, chat

7.3 Database / Storage

Table 7.3: *Database and storage*

Database / Storage	Version	Purpose	Data Managed
PostgreSQL	15/16	Primary relational store	Users, forms, questions, responses, teams, analytics, billing
Prisma Client	5.12	Type-safe ORM	Queries, transactions, migrations
Redis	7 (optional)	Cache and queue backend	Cached lookups, job queue; in-memory fallback when disabled
Local file storage	—	Uploads directory	Files uploaded in responses and documents
Browser localStorage	—	Client session	Access/refresh tokens, theme preferences

7.4 Hardware / Cloud / Deployment Environment

Table 7.4: *Hardware, cloud and deployment environment*

Platform / Hardware	Specification	Purpose	Status
Developer laptop	Windows 10, 8–16 GB RAM, SSD	Development and testing	Used
Docker Compose	PostgreSQL + Redis containers	Local infrastructure	Used
Vercel	Next.js hosting with CDN	Frontend production	Deployed
Render	Node web service (render.yaml)	Backend production	Deployed
Managed PostgreSQL	Cloud database	Production data	Configured
Google AI Studio	Gemini API key	LLM access	Configured

7.5 Development Tools

Table 7.5: *Development tools*

Tool	Purpose	Usage
Visual Studio Code	IDE	Editing, debugging, integrated terminal
Git	Version control	66 commits with descriptive messages on main branch
npm workspaces	Monorepo management	frontend, backend and packages/shared
nodemon + ts-node	Backend hot reload	npm run dev:backend
ESLint + Prettier	Code quality	Consistent formatting and linting
Prisma CLI / Studio	DB tooling	generate, migrate, seed, inspect data
Postman / custom Node scripts	API testing	Manual checks and automated QA runner
Chrome / Edge DevTools	Frontend debugging	Network, console, responsive checks

7.6 Technology Selection Rationale

TypeScript end-to-end was selected so that the same types and Zod schemas describe forms in the frontend, backend and shared package, catching mismatches at compile time. **Next.js** offers file-based routing, optimised production builds and first-class deployment on Vercel, and **React 19** provides a modern component model. **Tailwind CSS** enabled a consistent design system and dark mode with little custom CSS.

Express was chosen for the API because it is minimal, mature and easy to compose with security middleware; it also makes streaming responses (AI chat) straightforward. **PostgreSQL** provides relational integrity for users, teams and billing together with JSON columns for flexible form definitions, and **Prisma** adds type-safe queries and migrations.

Google Gemini was selected as the LLM because it supports multimodal input (images and PDFs) in the same request, native JSON output with response schemas, a generous free tier and several model sizes that allow a cost/latency-aware fallback chain. Vercel and Render were selected for hosting because both deploy directly from Git, provide HTTPS by default and have free tiers suitable for an internship project.

CHAPTER 8

SYSTEM IMPLEMENTATION

Technical core of the report; consolidate Reporting 2, 3 and 4 implementation work.

8.1 Implementation Overview

The system is implemented as an npm-workspaces monorepo with three packages. **frontend/** contains the Next.js application with 29 page routes (landing, auth, dashboard, AI generator, builder, templates, analytics, responses, team, settings, billing, admin, public form and others), reusable components (DashboardLayout, AssistantChat, notification panel, team components, UI primitives) and lib/api.ts, which wraps fetch with automatic token refresh. **backend/** contains the Express server (index.ts), 8 routers exposing 73 routes, 4 middlewares, 3 services and the lib folder with the AIFormBrain engine, document parser, cache, queue and validation schemas. **packages/shared/** holds shared types and schemas.

Development started with the database schema and authentication (Reporting 2), followed by forms CRUD, the builder and the public responder. The AI engine, document digitisation and quiz engine were implemented next (Reporting 3), followed by analytics, teams, notifications, billing and the admin panel. Reporting 4 concentrated on defect fixing, performance tuning, deployment and documentation. The three central modules are described in the following sections.

8.2 Module / Feature 1 – AI Form Generator

The AI generator is exposed as POST /api/ai/generate (and /generate-from-file). The route accepts a prompt, optional file (via Multer), desired question count and theme preferences. It first validates the input: prompts shorter than three characters return suggestions rather than an error, unauthenticated users can try generation in demo mode, and authenticated users must have at least 5 AI credits and must not exceed the free-plan limit of 5 active forms.

Uploaded documents are converted to text by documentParser (pdf-parse for PDF, mammoth for DOCX, SheetJS for spreadsheets), while images and PDFs are also passed to Gemini as inline base64 data for multimodal understanding. The AIFormBrain pipeline

then produces a deterministic plan, Gemini produces natural wording through a JSON response schema, and the EnterpriseQualityReviewer merges and repairs the result. The form is saved as PUBLISHED with a unique share id, 5 credits are deducted and the response includes an “understanding summary” (topic, purpose, form type, difficulty, question count) that is shown to the user. The AI Generator page is shown in Figure 8.1.

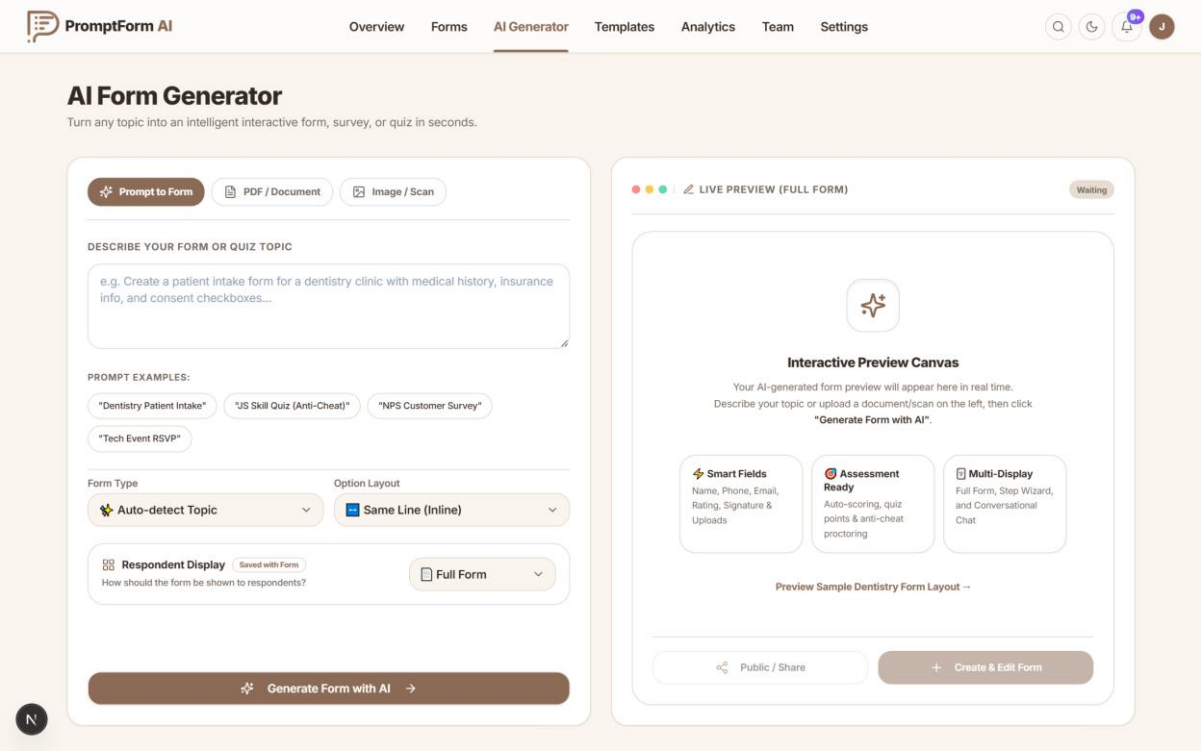


Figure 8.1: *Module 1 Implementation / Output*

8.3 Module / Feature 2 – Visual Form Builder

The builder (/builder/[id]) loads the form and renders a three-pane editor. The left palette groups field presets (Profile & Contact, Choices & Options, Advanced) such as Full Name, Email, Phone, Address, Country, Gender, Checkboxes, Radio, Agreement, Rating, Date, File Upload and Signature. The canvas shows the form exactly as respondents will see it; each question card supports duplicate, delete, reorder, logic and type badges. The right panel edits field properties (type, label, required, options, validation) and the live branding editor (accent colour, typography, corner radius, display mode).

State is held in a Zustand store with an undo/redo history. Saving calls PUT /api/forms/:id/questions, which upserts questions while preserving existing question UUIDs so that historical responses remain linked (a defect fixed during QA, see Section 9.9). Publish/share, preview, collaborators and close-form actions are available from the toolbar. Figure 8.2 shows the builder editing the AI-generated Patient Intake Form.

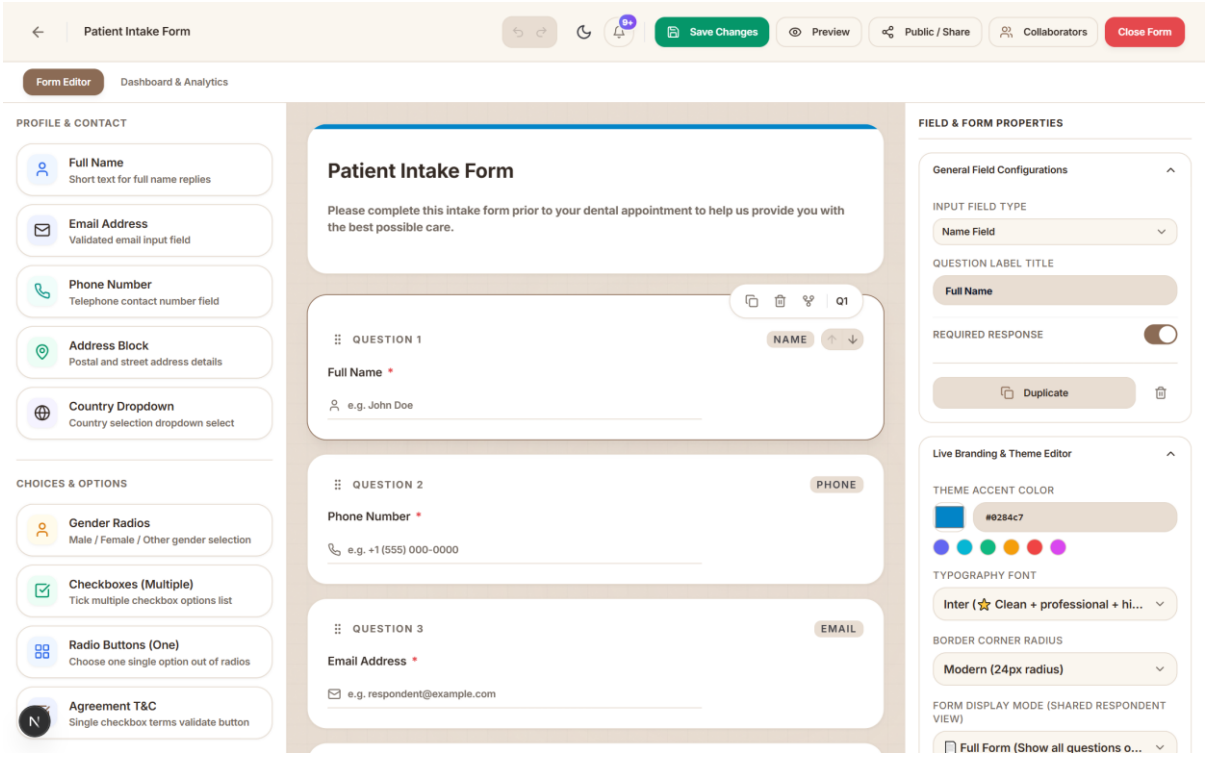


Figure 8.2: *Module 2 Implementation / Output*

8.4 Module / Feature 3 – Responses, Quiz Engine and Analytics

Public forms are served at /f/[shareId] in full, wizard or chat mode. The submit endpoint (POST /api/forms/:id/submit) is rate limited and validates answers with Zod, enforces required fields and form passwords on the server, applies response limits and auto-closes forms when the limit is reached. For quizzes, the EnterpriseQuizEngine attaches correct answers, points, negative points, difficulty and Bloom's taxonomy level to each question; the client enforces a timer and counts tab switches, and submissions exceeding the threshold are flagged in browserMetadata. Scores are calculated with weighted marks and exported to CSV for leaderboards.

Every view, start, submission and drop-out is recorded as a FormEvent with device (ua-parser-js) and country (geoip-lite). The analyticsService aggregates these into FormDailyMetric rows and the per-form dashboard (Figure 8.3) presents views, submissions, completion rate, average time, historical trends, device distribution, top locations and an AI sentiment summary of text answers.

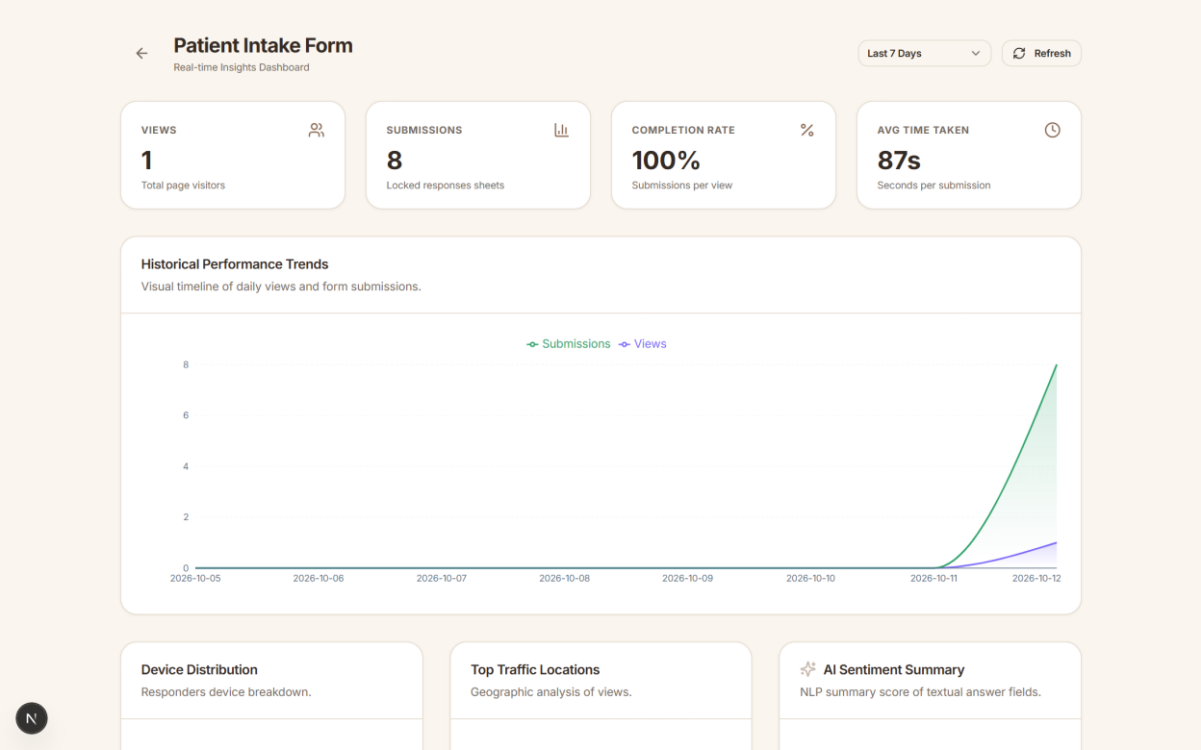


Figure 8.3: *Module 3 Implementation / Output*

8.5 Algorithm / Logic / Model Description

The heart of the system is the hybrid generation algorithm implemented in backend/src/lib/aiBrain. Its steps are:

1. **Intent detection:** normalise the prompt (lower-case, spelling and phonetic recovery), extract the requested action (create quiz, survey, registration…), desired question count and constraints.
2. **Language detection:** identify the language and right-to-left scripts so the theme can set rtl = true.
3. **Domain and industry detection:** score the prompt and any document context against keyword dictionaries for domains (quiz, survey, medical, education, HR, events, feedback, contact…) and industries (healthcare, dentistry, hospitality, education…).
4. **Requirement analysis and deep reasoning:** derive mandatory fields for the domain (e.g. patient name, date of birth and allergies for medical intake) and recommended settings (collect e-mails, response limits).
5. **Smart field intelligence:** map each planned field to one of the widget types and attach validations — e-mail pattern, phone pattern, numeric ranges, date pickers, file size and type limits.

6. **Form generation:** assemble title, description, ordered questions and an HSL-harmonised theme.
7. **Quiz enrichment:** for quiz, survey and medical domains, the quiz engine adds options, correct answers, explanations, points, negative points and difficulty, and enables shuffle, anti-cheat and a 600-second timer for quizzes.
8. **LLM generation:** the prompt, the plan and a system instruction are sent to Gemini with responseMimeType = application/json and a response schema. Models are tried in order (gemini-3.5-flash-lite → gemini-3.1-flash-lite → gemini-2.5-flash → gemini-flash-latest → gemini-2.5-pro) with a 45-second timeout each; invalid-key errors stop immediately, and if inline file data is rejected the request is retried as text only.
9. **Quality review:** EnterpriseQualityReviewer removes duplicate labels, fixes types that conflict with labels (e.g. an "Email" field typed as short text), ensures choice questions have at least two options and normalises ordering. If every model fails, the deterministic plan from steps 1–7 is returned, so the user always receives a usable form.

```
export class AIFormBrain {
  static process(prompt: string, documentContext?: string) {
    let context = IntentDetector.detect(prompt);
    context = LanguageDetector.detect(context);
    const domain = DomainDetector.detect(context, documentContext);
    const industry = IndustryDetector.detect(domain, context.normalizedPrompt);
    const requirements = RequirementAnalyzer.analyze(context, domain);
    const reasoning = DeepReasoner.reason(domain, requirements);
    const plan = SmartFieldIntelligenceEngine.resolve(domain, industry,
                  context.normalizedPrompt, documentContext);
    const formConfig = EnterpriseFormGenerator.generate(plan);
    if (domain === 'quiz' || domain === 'survey' || domain === 'medical') {
      const assessment = EnterpriseQuizEngine.resolve(domain,
                  context.normalizedPrompt, formConfig.questions);
      /* merge points, answers, difficulty; enable anti-cheat, timer */
    }
    formConfig.settings = { ...formConfig.settings, ...reasoning.recommendedSettings };
    formConfig.theme.rtl = context.isRtl;
    return EnterpriseQualityReviewer.reviewAndImprove(formConfig);
  }
}
```

Listing 8.1: Excerpt of the AIFormBrain pipeline (backend/src/lib/aiBrain/index.ts)

An abridged example of the response returned by the generator for the prompt "Patient intake form for a dental clinic with contact details, medical history and appointment date" is shown in Listing 8.2. The understanding summary is displayed to the user, and the form is immediately available at its share link.

```
{
  "message": "Form generated successfully using PromptForm AI engine.",
  "understandingSummary": {
    "topic": "Dentistry Clinic", "purpose": "Lead Generation & Communication Inquiry",
    "formType": "Contact Form", "difficulty": "Intermediate", "questionCount": 6
  },
```

```
  "form": {
    "title": "Patient Intake Form", "status": "PUBLISHED", "uniqueShareId": "f5nmxd",
    "settings": { "collect_emails": true, "anti_cheat_detection": false, "timer_limit": 0 },
    "theme": { "primary_color": "#0284c7", "background_color": "#f0f9ff", "font_family":
"Inter" },
    "questions": [
      { "type": "name",  "label": "Full Name",     "required": true },
      { "type": "phone", "label": "Phone Number",  "required": true },
      { "type": "email", "label": "Email Address", "required": true },
      { "type": "mcq",       "label": "...", "options": ["...", "..."] },
      { "type": "checkbox", "label": "...", "options": ["...", "..."] },
      { "type": "short_text", "label": "..." }
    ]
  }
}
```

Listing 8.2: Abridged generator response for the dental intake prompt

8.6 API / External Service Integration

Table 8.1: *API and external service integration*

API / Service	Purpose	Input	Output	Authentication	Status
POST /api/ai/generate	Generate form	Prompt / file	Form + summary	Optional JWT	Working
POST /api/ai/chat-stream	AI assistant edits	Message, form	Streamed reply	JWT + plan	Working
POST /api/ai/analyze-response(s)	AI insight / sentiment	Responses	Analysis JSON	JWT + plan	Working
/api/auth/*	Register, login, refresh, logout, OAuth, coupons	Credentials	Tokens, profile	Public / JWT	Working
/api/forms/*	CRUD, questions, collaborators, submit, export, notifications	Form JSON	Form, CSV	JWT / public submit	Working
/api/teams/*	Teams, members, invites, activity	Team data	Team JSON	JWT + RBAC	Working
/api/analytics/*	Workspace and form metrics, events	Form id, events	Metrics	JWT / public events	Working
Google Gemini generateContent	LLM generation	Prompt, schema, inline file	JSON text	API key	Working
Google OAuth 2.0	Social login	Auth code	Profile	Client id/secret	Working
Gmail SMTP / Resend	Transactional e-mail	Recipient, HTML	Message id / status	App password / API key	Working

8.7 Database Implementation

The schema is defined in backend/prisma/schema.prisma and applied with prisma migrate; prisma generate produces a typed client used through a single shared instance (lib/db.ts). UUID primary keys avoid enumeration of records. Foreign keys use onDelete: Cascade so that deleting a form removes its questions, responses, events and analytics, while a deleted team sets Form.teamId to null instead of deleting forms. Composite unique constraints prevent duplicate team membership and duplicate collaborators, and indexes on ownerId, teamId, formId, token, email and (formId, eventType, createdAt) keep dashboard queries fast.

JSON columns store flexible structures: Form.settings (collect_emails, limit_responses, password, shuffle_questions, timer_limit, anti_cheat_detection, display_mode), Form.theme, Question.options/validations/logic and Response.answers/browserMetadata. Multi-step operations such as accepting a team invitation (verify token, create membership, mark invite accepted) run inside Prisma transactions to remain atomic. A seed script creates an admin and a demo user and starter templates for development.

8.8 Security / Access Control (if applicable)

Security was implemented in layers following OWASP recommendations; the authentication flow is shown in Figure 8.4.

- **Password security:** minimum 10 characters, hashed with bcrypt (cost 10); passwords are never returned by the API.
- **Token-based authentication:** short-lived JWT access tokens and refresh tokens; the client refreshes automatically on 401.
- **Single active session:** every login stores a new activeSessionToken; tokens carrying an older session id are rejected with SESSION_MISMATCH, and logout clears the session.
- **Authorisation:** authMiddleware for protected routes, subscriptionMiddleware for plan limits and credits, and role checks (owner / admin / editor / viewer) for team and collaborator actions.
- **HTTP hardening:** Helmet security headers, CORS restricted to known frontend origins, JSON body limit of 20 MB.
- **Abuse protection:** rate limits of 1000 requests per 15 minutes for the API, 60 per minute for AI routes and 60 per minute for submissions in production.

- **Input validation:** Zod schemas on request bodies, server-side required-field and password enforcement on submissions, upload type/size restrictions.



Figure 8.4: *Authentication and Authorization Flow*

```
// Enforce single active session & invalidation on logout
if (!user.activeSessionToken || user.activeSessionToken !== decoded.sessionToken) {
  return res.status(401).json({
    error: 'SESSION_MISMATCH',
    message: 'You have been logged out because your account is active on another device.'
  });
}

const aiGenerationLimiter = rateLimit({ windowMs: 60 * 1000, max: 60 });
app.use('/api/ai', aiGenerationLimiter);
```

Listing 8.3: Session enforcement (middlewares/auth.ts) and AI rate limiting (index.ts)

8.9 Integration of Modules

Modules are integrated through the REST API and shared types. The frontend lib/api.ts attaches the bearer token, retries once after refreshing an expired token and normalises errors, so every page uses the same integration path. On the backend, routers delegate to services (formsService, analyticsService, emailService) and to the AIFormBrain, while cross-cutting concerns — authentication, plan limits, logging and error handling — are applied as Express middleware in a fixed order: Helmet → CORS → request logger → body parsers → rate limiters → routers → 404 → error handler.

Events connect otherwise independent modules: a submission creates a Response, records a FormEvent, updates analytics, writes an Activity entry, raises a live notification for the owner and, if configured, triggers a Workflow (e-mail or webhook). Team invitations integrate the teams module with the e-mail service and the authentication module: an invite token survives sign-up or login, the invitee e-mail is verified and membership is created atomically. A bootstrap sequence at server start validates environment variables, database connectivity, the uploads directory and route registration, and exposes /health, /health/liveness and /health/readiness endpoints for the hosting platform.

CHAPTER 9

TESTING AND VALIDATION

9.1 Testing Strategy

Testing combined manual exploratory testing during development with automated black-box API test suites executed against a running backend and real PostgreSQL database. The automated **platform QA suite** (backend/src/scripts/runTests.ts, executed with npm run test) contains 39 test cases across nine suites: Auth, Billing, Builder, Runner, Quiz, Analytics, Team, AI and Workflows. The **AI generator suite** contains 11 cases covering text, quiz, survey, PDF and image generation, question-count control, credit and plan limits, publication status and display-mode persistence. Results were written to qa_test_report.json and ai_generator_qa_report.json.

Each test specifies an input, the expected HTTP status or behaviour and the observed result. Defects discovered in the first run were logged, fixed and re-verified; these appear with status "Pass (fix verified)". Positive, negative and boundary cases were included (e.g. weak passwords, duplicate e-mails, 3 credits remaining, 6th form on the free plan). UI behaviour was validated manually in Chrome and Edge at desktop and mobile widths, and user acceptance testing was carried out with the mentor and peers.

9.2 Unit / Module Testing

Table 9.1: *Unit / module testing*

Test ID	Module / Function	Input	Expected	Actual	Status
UT-01	IntentDetector.detect	"quiz on photosynthesis 5 questions"	domain quiz, count 5	quiz, 5	Pass
UT-02	DomainDetector	Dental patient intake prompt	medical / dentistry	medical, dentistry	Pass
UT-03	Field intelligence	Label "Email Address"	type email + pattern	email	Pass
UT-04	Field intelligence	Label "Phone Number"	type phone	phone	Pass
UT-05	Quality reviewer	MCQ with one option	≥ 2 options	options filled	Pass
UT-06	LanguageDetector	Arabic prompt	rtl = true	true	Pass
UT-07	titleSanitizer	Prompt with "create a form for…"	Clean title	Clean title	Pass

Test ID	Module / Function	Input	Expected	Actual	Status
UT-08	documentParser	Sample PDF	Extracted text	Text returned	Pass
UT-09	Zod submit schema	Missing user_agent	Validation error	400 details	Pass
UT-10	Gemini fallback	First model unavailable	Next model used	Fallback used	Pass

9.3 Functional Testing

Table 9.2: *Functional testing (platform QA suite)*

Test ID	Functionality	Expected	Actual	Status
AUTH-01	User Registration (Valid Data)	201	201	Pass
AUTH-01-NEG	Registration Rejection on Weak Password (<10 chars)	400	400	Pass
AUTH-01-DUP	Registration Duplicate Email Guard	400	400	Pass
AUTH-02	User Login & JWT Grant	200	200	Pass
AUTH-02-NEG	Login Rejection on Wrong Password	400	400	Pass
AUTH-03	Single Active Session Enforcement	401 SESSION_MISMATCH	401	Pass
AUTH-04	Refresh Token Grant & Rotation	200	200	Pass
AUTH-05	Logout Session Token Invalidation	401 Unauthorized	401	Pass (fix verified)
AUTH-06	Google OAuth Sign-In Simulation	200	200	Pass
BILL-01	Free Plan 5-Form Creation Ceiling	403 LIMIT_REACHED	403	Pass
BILL-02	Slot Recovery upon Form Deletion	201	201	Pass
BILL-03	Upgrade to PRO Plan & Limit Removal	201	201	Pass
BILL-04	Enterprise Plan Recognition & Bypass	201 Created	201	Pass (fix verified)
BILL-05	Promo Coupon Validation & Discount Grant	200 (Valid) / 404 (Invalid)	200	Pass
BILL-06	User Transaction & Receipt Audit Trail	200	200	Pass
BLD-01	Form Creation & Retrieval by UUID	200	200	Pass
BLD-02	Bulk Question Insertion with Validations & Types	200	200	Pass
BLD-03	Question UUID Preservation on Save	Question IDs preserved	200	Pass (fix verified)
BLD-04	Conditional Logic JSONB Storage	200	200	Pass

9.4 Integration Testing

Table 9.3: *Integration testing*

Test ID	Integrated Components	Expected	Actual	Status
RUN-01	Public Resolution via uniqueShareId (Unauthenticated)	200	200	Pass
RUN-02	Server-Side Form Password Enforcement	403 without password, 201 with password	403	Pass (fix verified)
RUN-03	Server-Side Required Fields Validation	400 on empty, 201 on provided	400	Pass (fix verified)
RUN-04	Single Submission Constraint (limit_responses)	400	400	Pass
RUN-05	Form Response Limit & Auto-Close	400 CLOSED	400	Pass
RUN-06	Anti-Cheat Tab-Switch Tracking & Flagging	201 with flagged metadata	201	Pass
RUN-07	Public Form Anonymous File Upload	201 Created	201	Pass (fix verified)
QUIZ-01	Quiz Form Setup with Graded Questions	201	201	Pass
QUIZ-02	Quiz Responses CSV Export & Weighted Score Calculation	200 text/csv	200	Pass
QUIZ-03	Historical Response Preservation Across Form Updates	Historical answers preserved in CSV	200	Pass (fix verified)
ANL-01	Public View & Form Interaction Event Registration	200	200	Pass
ANL-02	Workspace Analytics Metrics & Time-Series Trends	200	200	Pass
ANL-03	Form-Specific Metrics Funnel & Dropout Analysis	200	200	Pass
TEAM-01	Workspace Team Creation & Member Invitation (Editor)	201	201	Pass
TEAM-02	RBAC Enforcement: Viewer Blocked from Modifying Form	403 Forbidden	403	Pass
TEAM-03	Team Owner Removal Protection Guard	400	400	Pass
TEAM-04	Team Rename (PUT) & Delete (DELETE) Endpoints	200 OK	200	Pass (fix verified)
WORKFLOW-01	Workflow Dispatch & Execution on Form Submit	201 Created and dispatched	201	Pass (fix verified)

9.5 System Testing

Table 9.4: *System testing (AI generator suite and end-to-end scenarios)*

Test ID	Scenario	Expected	Actual	Status
AI-01	AI Form Generation (Cognitive Brain Fallback)	200	200	Pass
AI-02	AI Credit Balance Enforcement (<5 credits required)	403 Insufficient credits	403	Pass
AI-GEN-01	Text Prompt Form Generation (Medical Intake)	200 OK with form and 5 credits deducted	Form "Dentistry Patient Knowledge Quiz" created with 5 questions. Credits deducted: 100 …	Pass
AI-GEN-02	Quiz Form Generation with Anti-Cheat & Graded Validations	200 OK with anti_cheat_detection=true and graded validations	Quiz generated with 5 questions. Anti-cheat enabled: true. Graded questions confirmed.	Pass
AI-GEN-03	Customer Survey Generation with Rating Scales	200 OK with rating widget field	Survey generated with 5 questions including rating widgets.	Pass
AI-GEN-04	PDF / Document Upload Form Generation	200 OK with parsed form	Generated form from PDF: "Extract Fields Dentistry Form" with 5 questions.	Pass
AI-GEN-05	Image / Scan Upload Form Generation	200 OK with generated form	Generated form from Image: "Education Event Registration & RSVP" with 10 questions.	Pass
AI-GEN-06	Question Count Configuration (3 questions target)	200 OK with around 3 questions	Form generated with 3 questions (target: 3).	Pass
AI-GEN-07	Insufficient Credit Balance Guard (< 5 credits)	403 Forbidden with Insufficient Credits message	Blocked generation when credits=3 with HTTP 403: "Insufficient credits. AI generation re…	Pass
AI-GEN-08	Short Prompt Suggester / Guard (<3 chars)	200 with suggestions or 400 validation error	Prompt "<3 chars" returned status='suggest' with suggestions: ["Product Review Form","Em…	Pass
AI-GEN-09	Free Plan 5-Form Ceiling Enforcement on AI Generation	403 Forbidden (LIMIT_REACHED)	subscriptionMiddleware properly blocked 6th AI form creation with HTTP 403: "Free plan l…	Pass

Test ID	Scenario	Expected	Actual	Status
AI-GEN-10	AI Generated Form Initial Status & Share Link Audit	Form created as PUBLISHED with uniqueShareId for immediate sharing	FIX VERIFIED: AI forms are created with status="PUBLISHED" and uniqueShareId="l7nm3s"! P…	Pass
AI-GEN-11	Display Mode (Full / Wizard / Chat) Persistence via PATCH /forms/:id	200 OK with settings.display_mode updated via PATCH	FIX VERIFIED: Endpoint PATCH /api/forms/:id successfully supported! display_mode="wizard…	Pass

9.6 Performance Testing

Table 9.5: *Performance testing*

Parameter	Condition	Target	Observed	Status
AI generation (text prompt)	Gemini flash-lite, warm server	< 10 s	≈ 4–9 s	Met
AI generation (document)	PDF / image upload	< 15 s	≈ 8–14 s	Met
Fallback generation	Gemini unavailable	< 1 s	< 300 ms (rule engine)	Met
Form CRUD API	Local PostgreSQL	< 500 ms	≈ 40–200 ms	Met
Public form load	Share link, warm	< 2 s	≈ 1 s	Met
Submission API	Valid payload	< 500 ms	≈ 100–250 ms	Met
Rate limiter	> 60 AI requests / min	HTTP 429	429 returned	Met
Cold start (Render free tier)	After 15 min idle	—	≈ 30–50 s first request	Known limitation

9.7 User / Acceptance Testing

Table 9.6: *User acceptance testing*

Scenario	Evaluator	Expected	Observed	Feedback / Status
Create event registration form from a prompt	Company mentor	Usable form in seconds	Generated in ≈ 6 s with correct types	Accepted
Digitise a scanned feedback form	Company mentor	Fields match paper form	Most fields matched; minor label edits	Accepted
Conduct a 10-question quiz	Peer students (5)	Timer, scoring, anti-cheat	Worked; tab switches flagged	Accepted
Edit theme and publish	Peer students	Intuitive builder	Completed without help	Accepted

Scenario	Evaluator	Expected	Observed	Feedback / Status
Invite team member as viewer	Institute mentor	Viewer cannot edit	Edit blocked (403)	Accepted
View analytics and export CSV	Company mentor	Correct counts and file	Counts matched; CSV opened in Excel	Accepted
Use on mobile phone	Peer students	Responsive pages	Responsive; wizard mode preferred	Accepted

9.8 Experimental Validation (if applicable)

Table 9.7: *Experimental validation of AI generation*

Experiment	Conditions	Result	Interpretation
LLM only vs hybrid	Same 11 prompts, schema output	Hybrid fixed wrong types and missing options	Rule review improves reliability
Model fallback chain	Primary model disabled	Generation still succeeded	Resilient to model outages
Question-count control	Target 3 questions	Exactly 3 generated	Count constraint respected
Multimodal input	Image of registration form	10 questions extracted	Image understanding usable
Short / vague prompts	< 3 characters	Suggestions returned	Graceful guidance instead of error

9.9 Defects / Bugs and Resolution

Table 9.8: *Defects identified and resolved*

Issue ID	Problem	Severity	Root Cause	Resolution	Final Status
AUTH-05	Token still valid after logout	High	Session not cleared on logout	Clear activeSessionToken on logout; middleware compares session id	Closed
BILL-04	Enterprise plan hit free limits	Medium	Plan check only recognised “pro”	Treat all paid plans as unlimited in subscription config	Closed
BLD-03	Question IDs regenerated on save	High	Delete-and-recreate on update	Upsert questions preserving UUIDs	Closed
RUN-02	Form password checked only in UI	High	No server check	Enforce password on submit endpoint (403)	Closed

Issue ID	Problem	Severity	Root Cause	Resolution	Final Status
RUN-03	Required fields bypassable via API	High	Client-only validation	Server-side required validation (400)	Closed
RUN-07	Anonymous file upload rejected	Medium	Upload route required auth	Allow public upload for public forms with limits	Closed
QUIZ-03	Old answers lost in CSV after edits	Medium	Export keyed by current questions	Preserve historical answers in export	Closed
TEAM-04	Team rename/delete missing	Low	Endpoints not implemented	Added PUT and DELETE /api/teams/:id	Closed
WF-01	Workflows not dispatched	Medium	Submit did not call workflow runner	Dispatch active workflows after submit	Closed
AI-GEN-10	AI forms created as DRAFT	Medium	Default status	Create AI forms as PUBLISHED with share id	Closed
UI-NOTIF	Notification panel under page content	Low	z-index / stacking context	Flex layout, viewport-bounded panel, Escape to close	Closed

9.10 Final Validation Statement

All 39 platform QA test cases and all 11 AI-generation test cases passed in the final run (50/50), including the nine defects that were fixed and re-verified (Figure 9.1). Functional, integration, system, performance and acceptance testing confirm that PromptForm AI meets the functional requirements FR-01 to FR-17 and the non-functional targets listed in Chapter 4, with the single known limitation of cold-start latency on the free hosting tier. The system is therefore validated as complete and ready for demonstration and use.

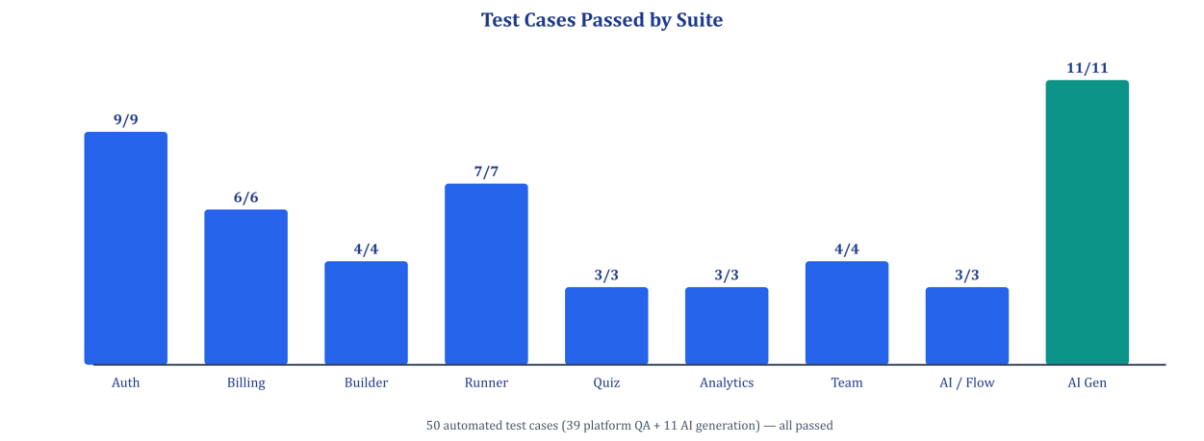


Figure 9.1: *Test Cases Passed by Suite*

CHAPTER 10

RESULTS AND PERFORMANCE ANALYSIS

10.1 Final Results

The final outcome of the project is a working, deployed SaaS platform. A user can register, describe a form in one sentence or upload a document, and within seconds receive a published form with a short share link (for example /f/f5nmxd), correctly typed fields, validations and a harmonised theme. During the final demonstration the prompt “Patient intake form for a dental clinic with contact details, medical history and appointment date” produced a six-question Patient Intake Form with name, phone, e-mail, choice and checkbox fields, and the prompt “Customer satisfaction survey for a restaurant” produced a six-question Restaurant Dining Experience Form with multiple-choice and long-text feedback questions.

The forms were edited in the builder, opened through the public link and filled by sample respondents; the dashboard immediately reflected 8 submissions per form with a 100% completion rate and an average time of 87 seconds, and the responses ledger listed each submission with time taken, tab switches and flag status. All 50 automated test cases passed. The application runs locally through a single command (node dev.js) and in production on Vercel and Render.

10.6 Results Discussion

The results confirm the central hypothesis of the project: combining deterministic domain rules with schema-constrained LLM generation produces forms that are both fast to create and reliable enough for real use. The most visible gain is time — form creation dropped from tens of minutes to seconds — but the quality gain is equally important. Because the field intelligence engine and quality reviewer enforce types and validations, AI-generated forms contained correctly typed e-mail and phone fields and complete option lists, which are frequent sources of error in manually built forms.

The fallback design proved valuable in practice. During development the Gemini API occasionally returned quota or availability errors; in those cases the next model in the chain or the rule-based plan was used, so the user never saw a failed generation. The security fixes found during QA (server-side password and required-field enforcement,

session invalidation on logout) show the value of testing at the API level rather than only through the UI.

Two limitations were observed. First, the first request after inactivity on the free hosting tier suffers a cold start of around 30–50 seconds, which affects perceived performance but not functionality. Second, documents with heavy handwriting or complex multi-column layouts can produce partially correct field lists that need manual correction in the builder. Both are discussed further in Chapter 13.

10.2 Objective-wise Achievement

Table 10.1: *Objective-wise achievement*

Objective	Achievement / Evidence	Status
O1 – Prompt to form in < 10 s	Average 4–9 s; AI-GEN-01 to 03 passed	Achieved
O2 – Digitise documents	PDF, DOCX, XLSX, image; AI-GEN-04, 05 passed	Achieved
O3 – ≥ 10 field types with validation	11 core types + contact variants; BLD-02 passed	Achieved
O4 – Visual builder	Three-pane builder with undo/redo and theme editor (Figure 8.2)	Achieved
O5 – Instant publish and share	Unique share id; RUN-01, AI-GEN-10 passed	Achieved
O6 – Graded quizzes with anti-cheat	QUIZ-01 to 03, RUN-06 passed	Achieved
O7 – Analytics and export	Dashboards, CSV export; ANL-01 to 03 passed	Achieved
O8 – Team collaboration	Roles, invites, RBAC; TEAM-01 to 04 passed	Achieved
O9 – Security controls	JWT, single session, bcrypt, Helmet, CORS, rate limits; AUTH suite passed	Achieved
O10 – ≥ 40 tests and cloud deployment	50 tests passed; Vercel + Render deployment	Achieved

10.3 Performance Metrics

Table 10.2: *Performance metrics*

Parameter / Metric	Target	Final Result	Difference / Improvement	Status
Form creation time (10 fields)	< 60 s	≈ 8 s (text), ≈ 12 s (document)	≈ 99% less than ≈ 20 min manual	Met
AI generation success rate	≥ 95%	100% (11/11 AI tests)	Fallback ensures a form is always returned	Met
Automated test pass rate	100%	50 / 50	9 defects fixed and verified	Met

Parameter / Metric	Target	Final Result	Difference / Improvement	Status
Correct field-type inference	≥ 90%	All typed contact fields correct in tests	Quality reviewer repairs conflicts	Met
CRUD API latency	< 500 ms	≈ 40–200 ms	2.5–12× better than target	Met
Submission latency	< 500 ms	≈ 100–250 ms	Within target	Met
Pages implemented	20	29 routes	+9 pages	Exceeded
REST endpoints	50	73 routes	+23 endpoints	Exceeded

10.4 Comparative Results (if applicable)

Indicative timings for producing a ready-to-share 8–10 field form were measured during the project (manual builder vs PromptForm AI); the comparison is summarised in the table below and charted in Appendix E.

Table 10.3: *Comparative results with existing approaches*

Method / System	Metric 1: Creation Time	Metric 2: Field-type Errors	Metric 3: Document Import	Observation
Manual builder (Google Forms)	≈ 15–25 min	Common (manual choice)	Not supported	Baseline
Template-based (Jotform)	≈ 5–8 min	Fewer, needs edits	Separate PDF tool	Faster but still manual
PromptForm AI – text prompt	≈ 8 s	Rare (auto-typed)	—	Fastest, consistent
PromptForm AI – document	≈ 12 s	Rare	PDF, DOCX, XLSX, image	Unique capability

10.5 Final Output Screenshots

The following screenshots were captured from the running application with the demo account and demonstrate the main outputs of the system.

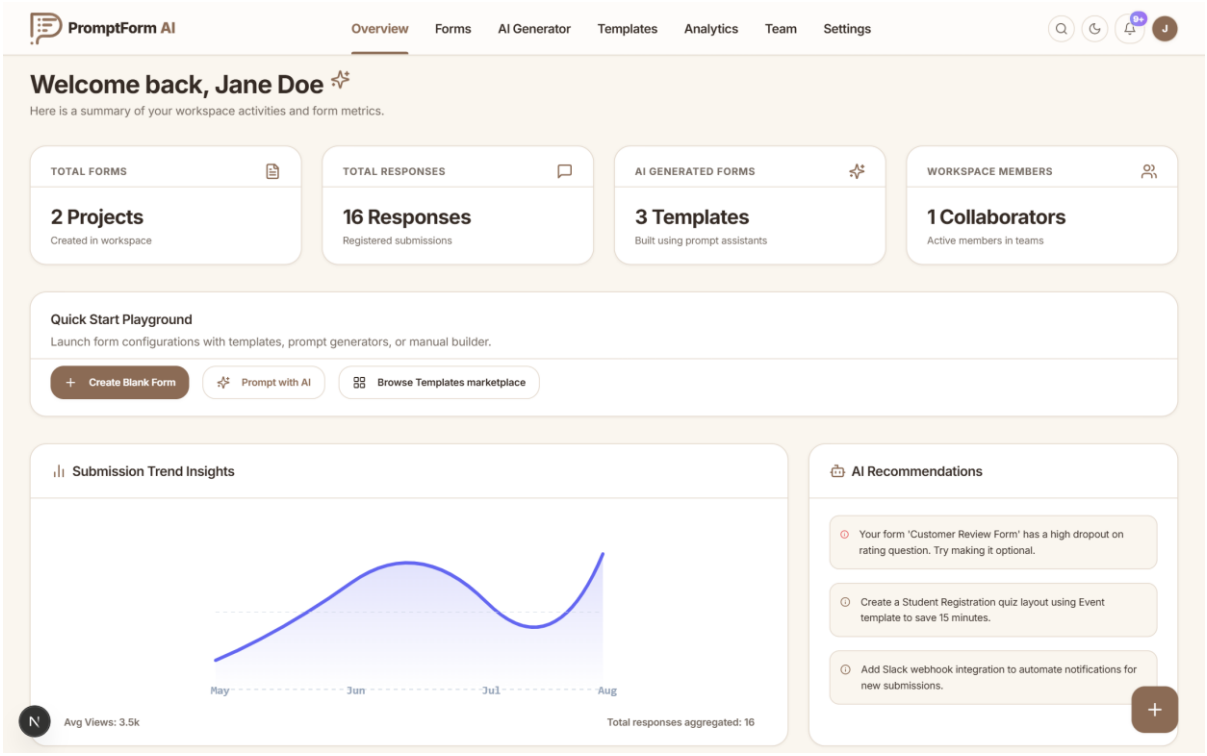


Figure 10.1: *Final Output / Result Screenshot 1 (Workspace Overview Dashboard)*

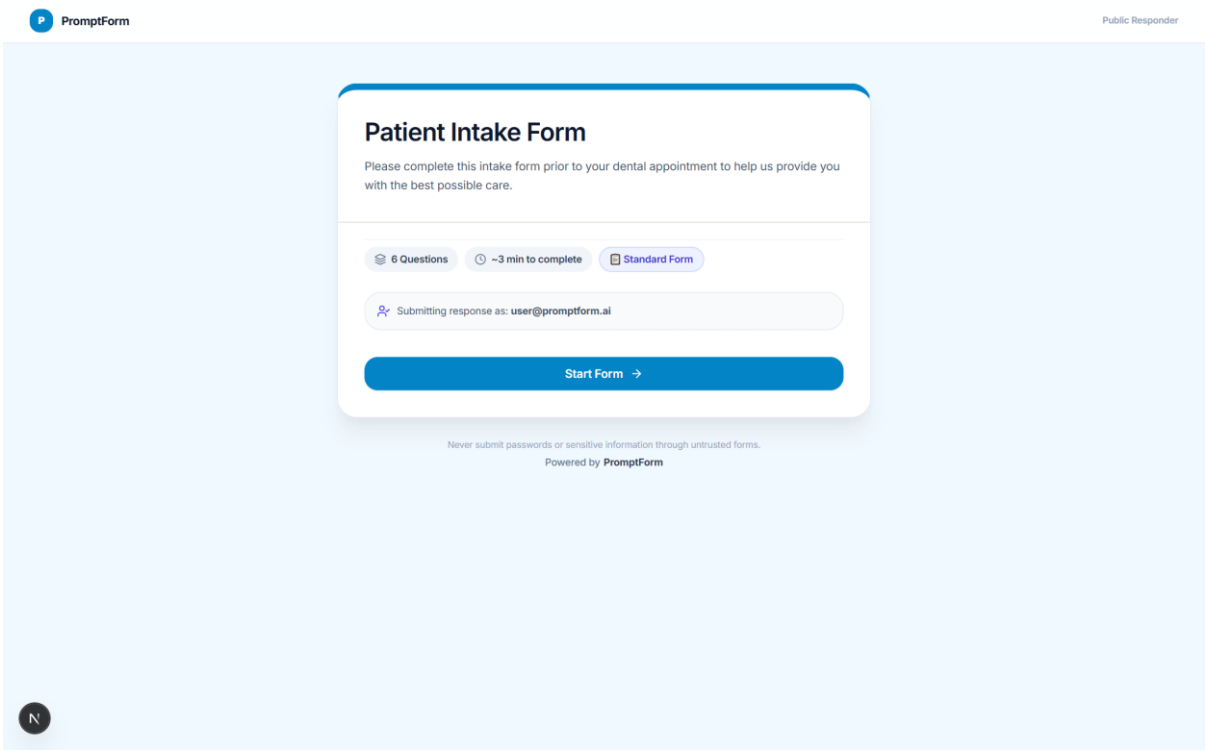


Figure 10.2: *Final Output / Result Screenshot 2 (Public Responder)*

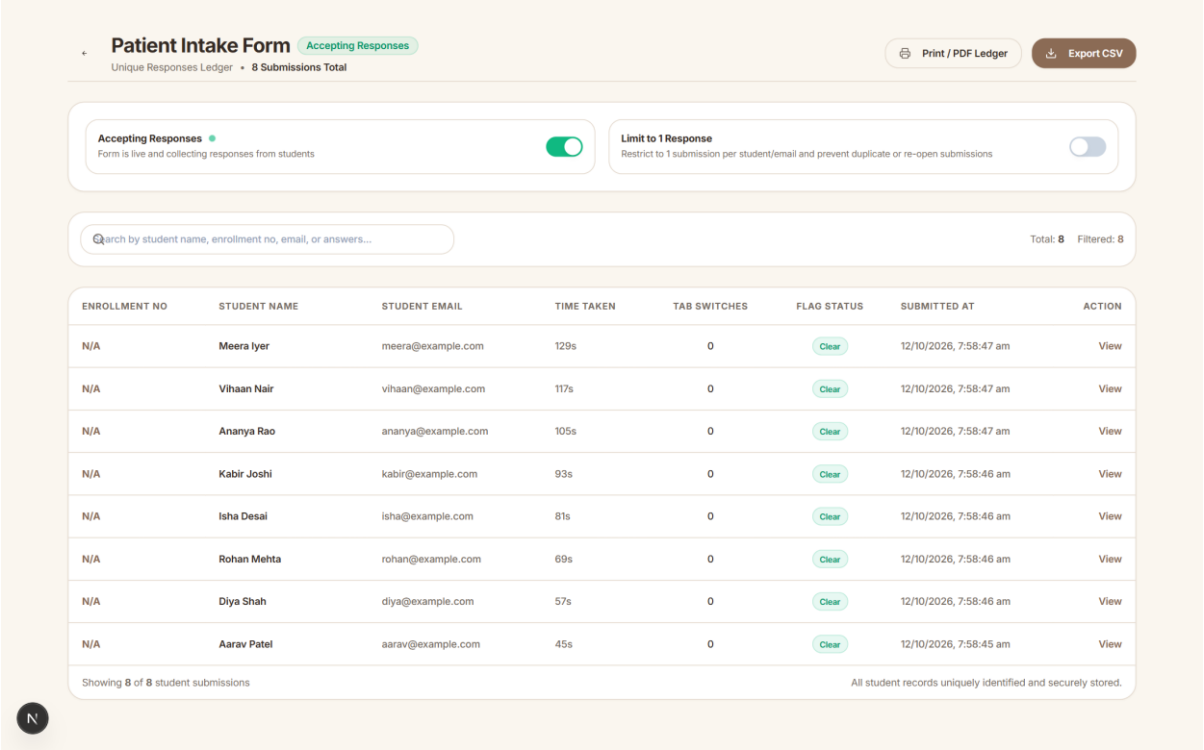


Figure 10.3: *Final Output / Result Screenshot 3 (Responses Ledger)*

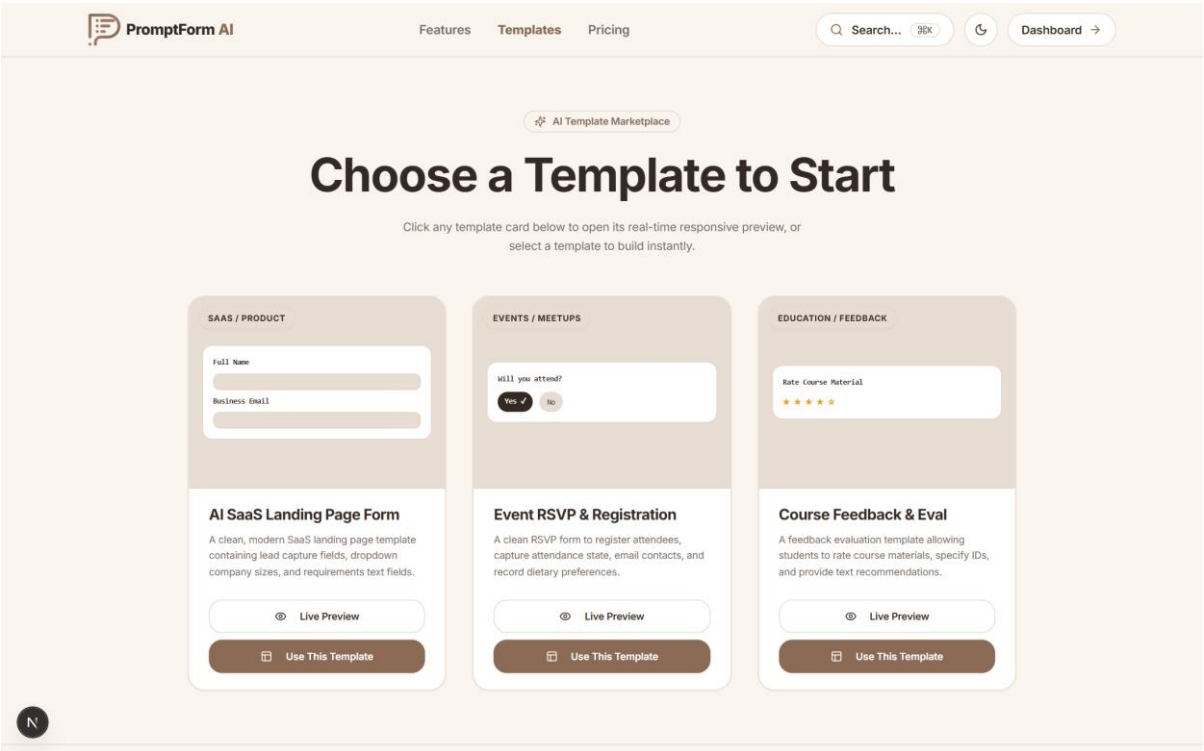


Figure 10.4: *Final Output / Result Screenshot 4 (Template Marketplace)*

CHAPTER 11

CHALLENGES, SOLUTIONS AND IMPROVEMENTS

11.1 Major Challenges

Table 11.1: *Major challenges*

Challenge	Phase	Impact	Solution	Final Outcome
Malformed or inconsistent LLM output	AI engine	Broken forms, wrong types	JSON response schema + quality reviewer	Consistent output
Gemini quota / model unavailability	AI engine	Failed generations	5-model fallback chain + rule-based plan	Always returns a form
Long AI latency	AI engine	Poor UX	Flash-lite models first, 45 s timeout, progress UI	≈ 4–9 s typical
Question IDs lost on save	Builder	Responses detached	Upsert with preserved UUIDs	History intact
Client-only validation	Testing	Security bypass	Server-side enforcement	Verified fix
Team invite flow across sign-up	Teams	Invites lost	Persist pending invite, verify e-mail, atomic membership	Reliable invites
Prisma client / port conflicts	Deployment	Server crashes	postinstall prisma generate, explicit PORT handling	Stable startup
E-mail deliverability	Notifications	Silent failures	Resend provider with SMTP fallback, domain checks	Honest delivery status

11.2 Technical Challenges

The most significant technical challenge was making LLM output dependable. Early prototypes asked the model for "a form in JSON" and parsed the text; responses sometimes contained markdown fences, missing fields, unsupported types or choice questions without options. The solution was to request native JSON with a response schema, validate the result, and pass it through the quality reviewer, which repairs types based on labels and fills missing options. Prompts were also enriched with the deterministic plan so that the model refines a sensible structure instead of inventing one from scratch.

A second challenge was multimodal document handling. Some models rejected certain inline file types, and large files increased latency. The implementation normalises MIME types, extracts text locally first (pdf-parse, mammoth, SheetJS) and, if a model rejects inline data, retries the request with text only. A third challenge was keeping analytics queries fast as events grew; this was addressed with a composite index on (formId, eventType, createdAt) and pre-aggregated FormDailyMetric rows.

11.3 Process / Coordination Challenges

Scope growth was the main process challenge: mentor feedback added document digitisation, quizzes and team collaboration to the original plan. MoSCoW prioritisation and sprint planning kept the work manageable — "must" features were completed first and "could" features were scheduled only after the core was stable. Coordinating reviews with two mentors (company and institute) required concise weekly demonstrations and written progress summaries, which later formed the basis of the four progress reports and this final report.

11.5 Lessons from Challenges

The challenges taught that AI features must be engineered like any other external dependency: with schemas, validation, timeouts, retries and a fallback path. They also showed that security must be enforced on the server, because API-level tests found issues that were invisible in the UI. Finally, regular demonstrations and a prioritised backlog proved essential for controlling scope while still incorporating valuable feedback.

11.4 Improvements Made During Development

Table 11.2: *Improvements made during development*

Improvement	Reason	Implementation	Impact
Hybrid AI engine	LLM-only output unreliable	AIFormBrain plan + Gemini + reviewer	Reliable, fast generation
Display modes	Long forms tiring on mobile	Full / wizard / chat responder	Better completion
Understanding summary	Users unsure what AI understood	Topic, purpose, type, difficulty shown	Transparency
Single active session	Shared-account abuse	Session token in JWT and DB	Stronger security
Live notifications	Owners missed responses	Notification panel + e-mail	Faster follow-up

Improvement	Reason	Implementation	Impact
Health endpoints & bootstrap checks	Opaque deployment failures	/health, readiness, env/DB validation	Easier operations
Notification panel polish	Layering and overflow bugs	Flex layout, viewport bounds, Escape to close	Usable on mobile

CHAPTER 12

INDIVIDUAL CONTRIBUTION AND PROFESSIONAL LEARNING

12.1 Individual Contribution

This was an individual internship project; all modules were designed, implemented, tested and deployed by the student under the guidance of the company and institute mentors.

Table 12.1: *Individual contribution*

Student	Module / Task	Contribution	Evidence	% Contribution
[STUDENT NAME]	Requirements & design	FR/NFR, architecture, UML, ER schema	Ch. 4, 6	100%
[STUDENT NAME]	Frontend	29 Next.js pages, components, API client	frontend/src	100%
[STUDENT NAME]	Backend & security	73 routes, middlewares, services	backend/src	100%
[STUDENT NAME]	AI engine	AIFormBrain, Gemini integration, document parsing	lib/aiBrain	100%
[STUDENT NAME]	Testing	50 automated tests, defect fixing	QA JSON reports	100%
[STUDENT NAME]	Deployment & documentation	Vercel/Render, reports	render.yaml, this report	100%

12.2 Technical Skills Acquired

- **Programming / Development:** advanced TypeScript, React 19 hooks and component design, Node.js and Express middleware patterns.
- **Frameworks / Libraries:** Next.js App Router, Tailwind CSS, Zustand, React Hook Form, Zod, Recharts, Prisma.
- **Database / Data Handling:** relational modelling, JSON columns, indexes, cascades, transactions and migrations in PostgreSQL.
- **Testing / Debugging:** API-level black-box test automation, defect triage, reproducing and verifying fixes.
- **Deployment / Cloud:** Vercel and Render deployment, environment configuration, health checks, Docker Compose.

- **Domain-specific skills:** prompt engineering, structured LLM output, multimodal document understanding and fallback design.

12.3 Professional Skills Acquired

- **Communication:** presenting progress to technical and academic mentors and explaining design trade-offs.
- **Teamwork / Collaboration:** incorporating reviewer feedback and following organisational coding conventions.
- **Time Management:** sprint planning and MoSCoW prioritisation within a fixed internship timeline.
- **Problem Solving:** diagnosing failures across frontend, backend, database and external services.
- **Documentation:** writing progress reports, test reports and this consolidated final report.
- **Presentation / Technical Explanation:** live demonstrations of AI generation, analytics and security features.

12.4 Major Learning Outcomes

The internship provided end-to-end experience of building a production-style product: translating a business problem into requirements, designing a scalable architecture, implementing a large TypeScript codebase, integrating an external AI service, securing it, testing it systematically and deploying it to the cloud. The most important learning was how to use generative AI responsibly in software — not as a black box, but as one component surrounded by validation, deterministic rules and graceful degradation.

The project also strengthened professional habits: writing meaningful commit messages, keeping the main branch deployable, documenting decisions and measuring results instead of relying on impressions. These outcomes directly support future work as a full-stack or AI application engineer.

CHAPTER 13

LIMITATIONS AND FUTURE SCOPE

13.1 Current Limitations

Table 13.1: *Current limitations*

Limitation	Reason	Impact	Possible Improvement
Dependence on Gemini API	Third-party LLM	Quota / outage reduces wording quality	Add second provider, caching, self-hosted model
Cold start on free hosting	Render free tier sleeps	30–50 s first request	Paid instance or keep-alive pings
No real payment settlement	Out of scope	Plans recorded but not charged	Integrate Razorpay / Stripe
Handwritten / complex documents	Model limits	Partial field extraction	Dedicated OCR and layout analysis
Conditional logic not auto-generated	Complexity	Logic set manually	Generate branching rules from prompt
Default JWT secret fallback in code	Developer convenience	Risk if env not set	Fail fast when JWT_SECRET missing
Web-only client	Time constraint	No offline use	PWA / native apps

13.2 Future Scope

- Payment gateway integration (Razorpay / Stripe) with subscriptions, invoices and paid forms.
- AI-generated conditional logic, calculated fields and multi-page sections.
- Voice-to-form and multilingual generation with automatic translation of published forms.
- Integrations with Google Sheets, Slack, Microsoft Teams, Zapier and webhooks with retry queues.
- Advanced quiz proctoring (webcam snapshots, full-screen enforcement) and certificate generation.
- Accessibility (WCAG 2.2 AA) audit and improvements, including screen-reader optimised responder.
- Progressive Web App with offline response capture and later synchronisation.

13.3 Scalability / Extension Opportunities

Because the API is stateless and authenticated with JWTs, it can be scaled horizontally behind a load balancer. The existing Redis and BullMQ integration can move AI generation, e-mail sending and analytics aggregation into background workers, keeping request latency low under load. PostgreSQL read replicas and partitioning of the FormEvent table by month would support high-volume analytics, and a CDN already serves the frontend globally.

The modular AIFormBrain allows new domains and industries to be added as rule modules without changing the API, and the provider-agnostic generation function can be extended to other LLMs. Multi-tenant enterprise features — SSO, audit export, data residency and custom domains for public forms — are natural extensions of the current team and activity models.

CHAPTER 14

CONCLUSION

The project began with a simple observation: building online forms is slow, repetitive and error-prone, and existing tools leave the entire authoring effort to the user. The objective was to create a platform that turns a natural-language description or an existing document into a complete, validated and shareable form in seconds, and that supports the rest of the form life cycle — responses, quizzes, analytics and collaboration.

An Agile methodology with two-week sprints and four progress milestones guided the work. Requirements were captured as 17 functional and 12 non-functional requirements, and the system was designed as a three-tier architecture documented with use case, data flow, class, sequence, activity, deployment and ER diagrams. The implementation — a TypeScript monorepo with a Next.js 15 frontend, an Express REST API with 73 routes, PostgreSQL through Prisma with 16 models, and a hybrid AIFormBrain + Google Gemini engine — delivers prompt and document based generation, a visual builder, public responders in three display modes, a quiz engine with anti-cheat, analytics, teams, billing, notifications and an admin panel.

Testing with 50 automated cases, all passing after nine defects were fixed and verified, together with performance measurements and acceptance testing by mentors and peers, confirms that the objectives were achieved. Form creation time fell from roughly 20 minutes to under 10 seconds while field-type accuracy improved. The final achievement is a deployed, secure and extensible product that demonstrates how generative AI can be integrated reliably into real software, and that provides a strong foundation for payments, conditional logic generation and multilingual features in the future.

14.1 Final Project Status

Table 14.1: *Final project status*

Area	Final Status	Evidence / Remarks
Requirements	Complete	FR-01 to FR-17 and NFR-01 to NFR-12 implemented
Design	Complete	Architecture, UML and ER diagrams (Chapter 6)
Implementation	Complete	Frontend, backend, AI engine, database (Chapter 8)
Testing	Complete	50/50 automated tests passed (Chapter 9)

Area	Final Status	Evidence / Remarks
Deployment	Complete	Vercel (frontend), Render (backend), managed PostgreSQL
Documentation	Complete	Four progress reports and this final report
Demonstration	Complete	Live demonstration to company and institute mentors

CHAPTER 15

REFERENCES

References are consistent with the progress reports and follow the APA 7th Edition style. Every listed source was used and is cited in the text. The references include:

- Research papers / journal articles
- Books
- Official documentation
- Technical websites
- Standards
- APIs / libraries
- Software documentation

15.1 References

[1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.

[3] Wroblewski, L. (2008). Web form design: Filling in the blanks. Rosenfeld Media.

[4] Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

[5] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

[6] OWASP Foundation. (2021). OWASP Top 10: 2021. https://owasp.org/Top10/

[7] Google. (2024). Gemini API documentation: Generate structured output. Google AI for Developers. https://ai.google.dev/gemini-api/docs/structured-output

[8] Vercel. (2024). Next.js documentation (App Router). https://nextjs.org/docs

[9] Prisma Data. (2024). Prisma ORM documentation. https://www.prisma.io/docs

[10] OpenJS Foundation. (2024). Express 4.x API reference. https://expressjs.com/en/4x/api.html

[11] The PostgreSQL Global Development Group. (2024). PostgreSQL documentation. https://www.postgresql.org/docs/

[12] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

[13] Meta Open Source. (2024). React documentation. https://react.dev/

[14] Schwaber, K., & Sutherland, J. (2020). The Scrum guide. https://scrumguides.org/

15.2 Citation Requirements

- Sources are cited in the text where external ideas, methods or technical information are used (e.g. Vaswani et al., 2017; Brown et al., 2020; Wroblewski, 2008; OWASP, 2021; Google, 2024).
- No fake, unrelated or unused references are included.
- Primary, official and reliable academic and technical sources are preferred.
- One consistent citation style (APA 7th Edition) is maintained.

CHAPTER A

APPENDICES

Appendix A – Progress Report 1 Summary

Reporting 1 covered problem identification and planning. The student studied existing form builders (Google Forms, Typeform, Jotform, Microsoft Forms), measured manual form creation time, and defined the problem statement and ten objectives. An Agile methodology was selected, the technology stack (Next.js, Express, PostgreSQL, Prisma, Gemini) was proposed, and a 14-week schedule was prepared. Mentor feedback recommended adding document upload and assessment support, which were added to the backlog.

Appendix B – Progress Report 2 Summary

Reporting 2 covered design and initial implementation. The three-tier architecture, UML diagrams and the Prisma schema were finalised. The monorepo was set up with frontend, backend and shared packages. Authentication (register, login, JWT, refresh, Google OAuth), forms CRUD, the first version of the visual builder, the public responder and the template marketplace were implemented and demonstrated. The first AI generation prototype using Gemini was also shown.

Appendix C – Progress Report 3 Summary

Reporting 3 covered advanced implementation, testing and validation. The AIFormBrain hybrid engine, multi-model fallback, document digitisation, quiz engine with anti-cheat, analytics dashboards, teams with RBAC and invitations, billing with credits and coupons, notifications and the admin panel were completed. The 39-case platform QA suite and the 11-case AI suite were executed; nine defects were logged and fixed.

Appendix D – Reporting 4 / Final Evaluation

Reporting 4 covered final completion and evaluation. All defects were re-verified (50/50 tests passing), performance was measured, the notification panel and team invite flow were polished, e-mail delivery was improved with the Resend provider, and the application was deployed to Vercel and Render with a managed PostgreSQL database.

The system was demonstrated end-to-end to the mentors and the final report was prepared.

Appendix E – Additional Screenshots / Outputs

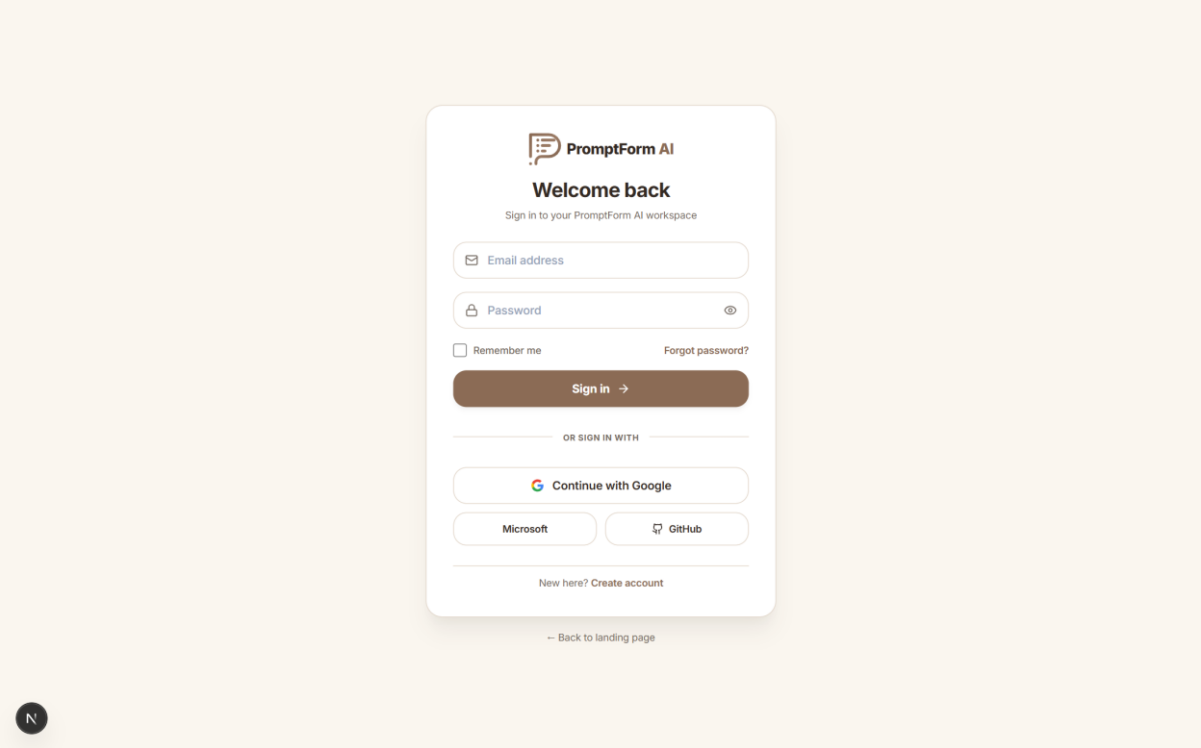


Figure A.1: *Supporting Evidence — Login Page*

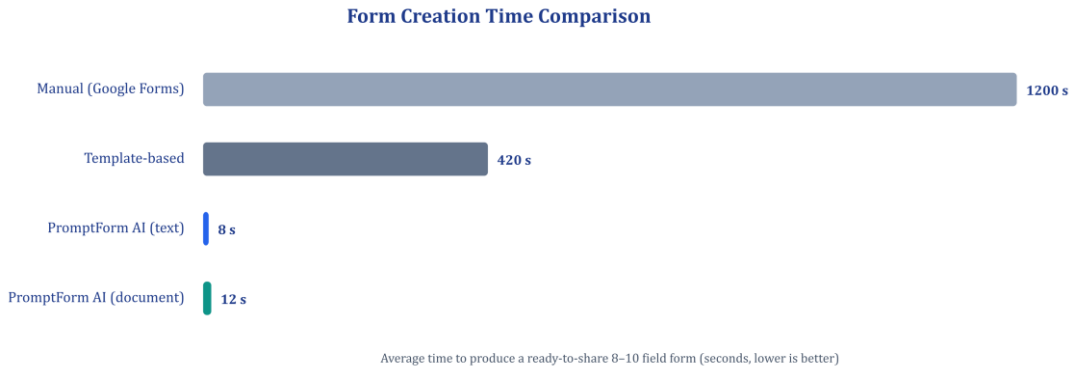


Figure A.2: *Supporting Evidence — Form Creation Time Comparison*

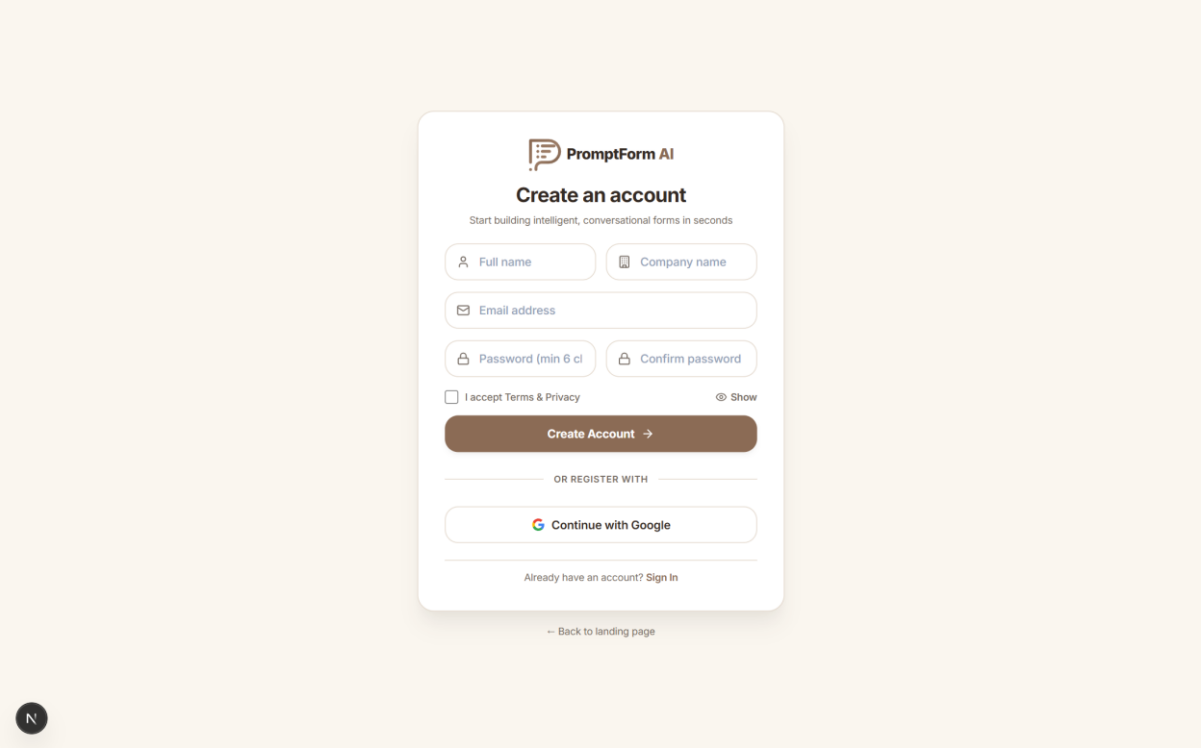


Figure A.3: *Supporting Evidence — Registration Page*

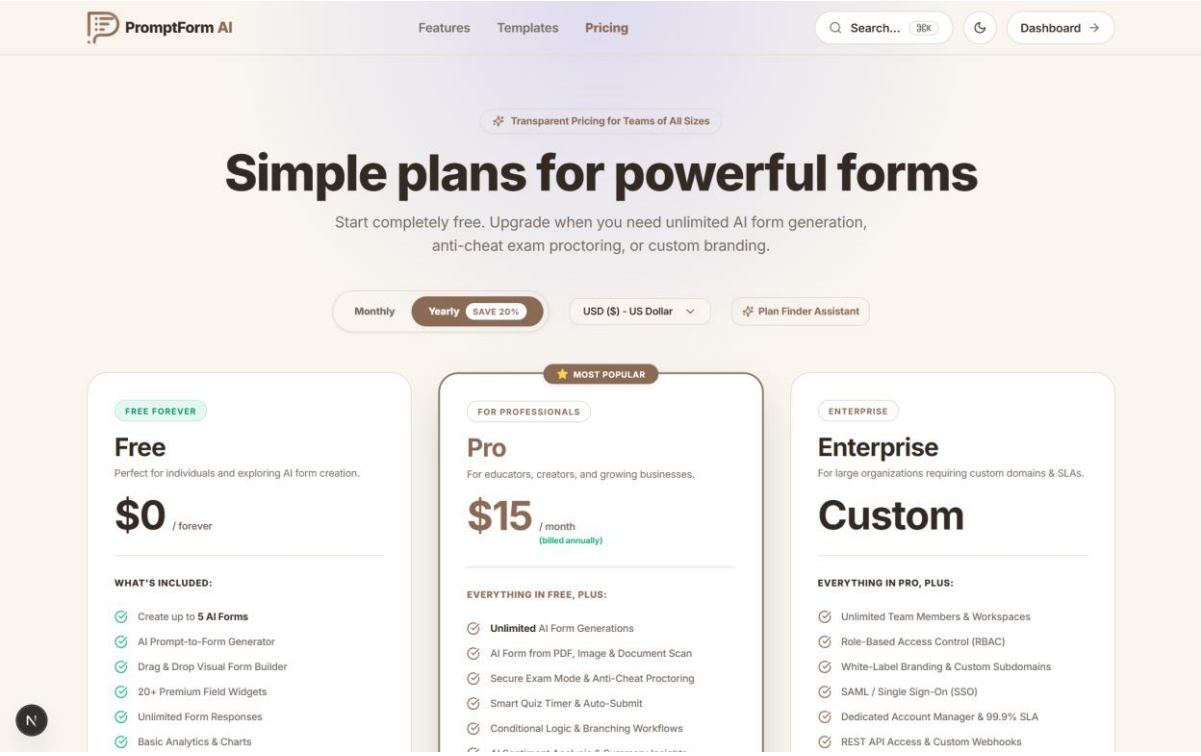


Figure A.4: *Supporting Evidence — Pricing Plans*

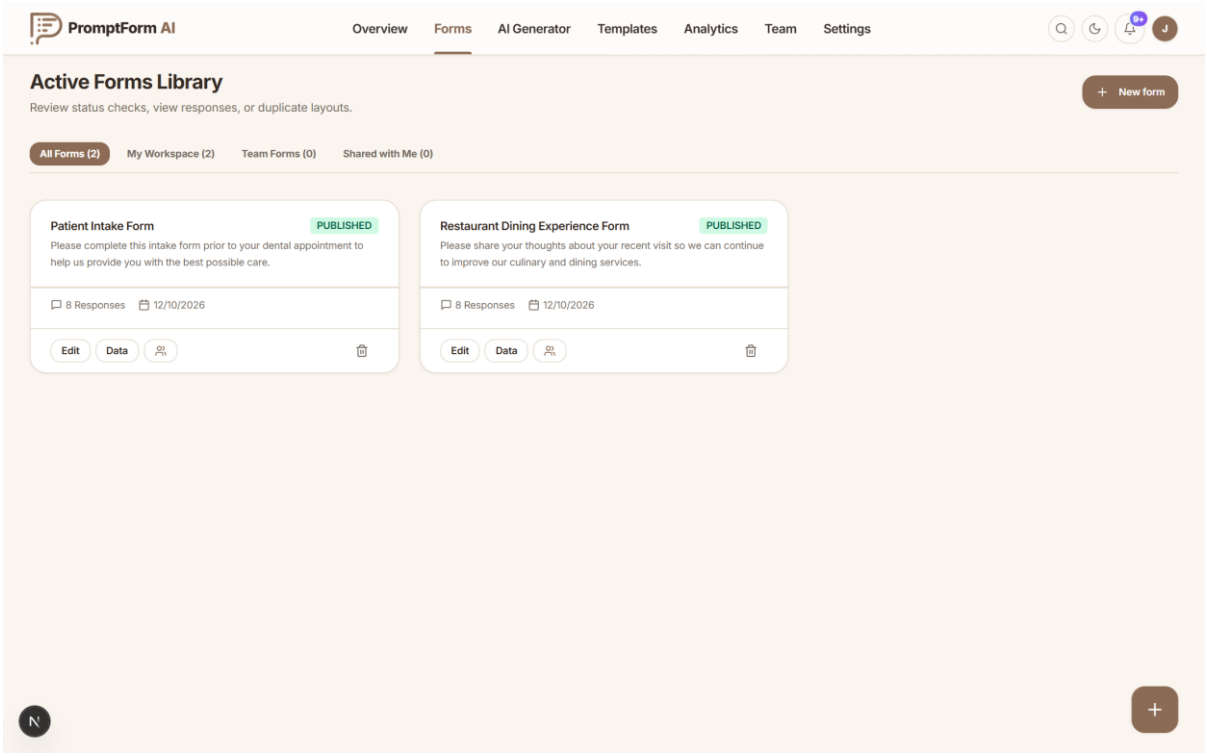


Figure A.5: *Supporting Evidence — Forms Library*

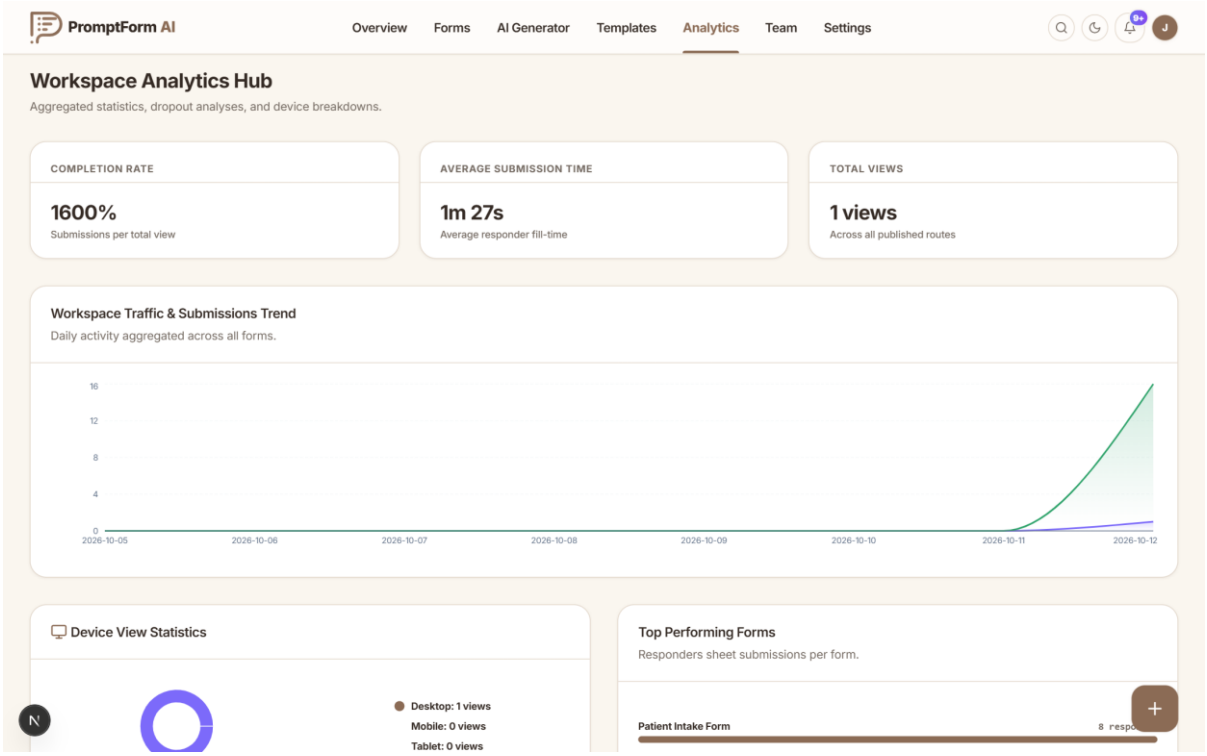


Figure A.6: *Supporting Evidence — Workspace Analytics*

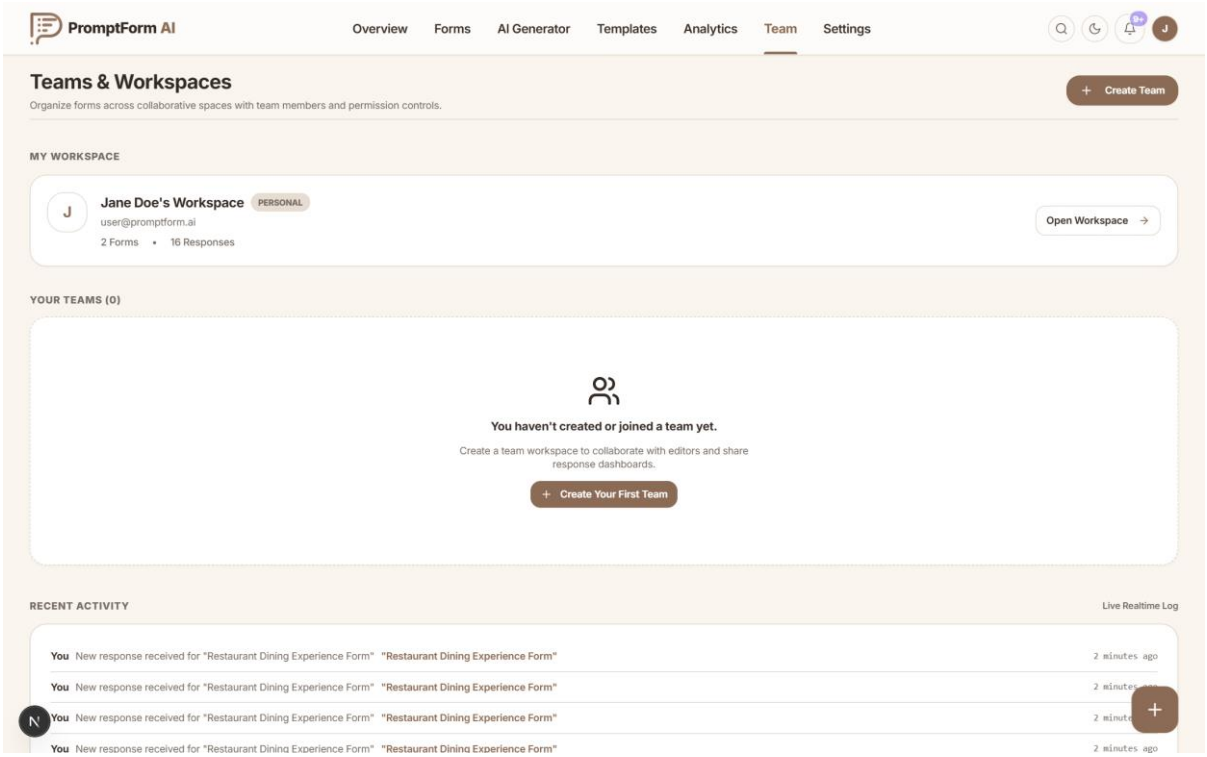


Figure A.7: *Supporting Evidence — Team Workspace*

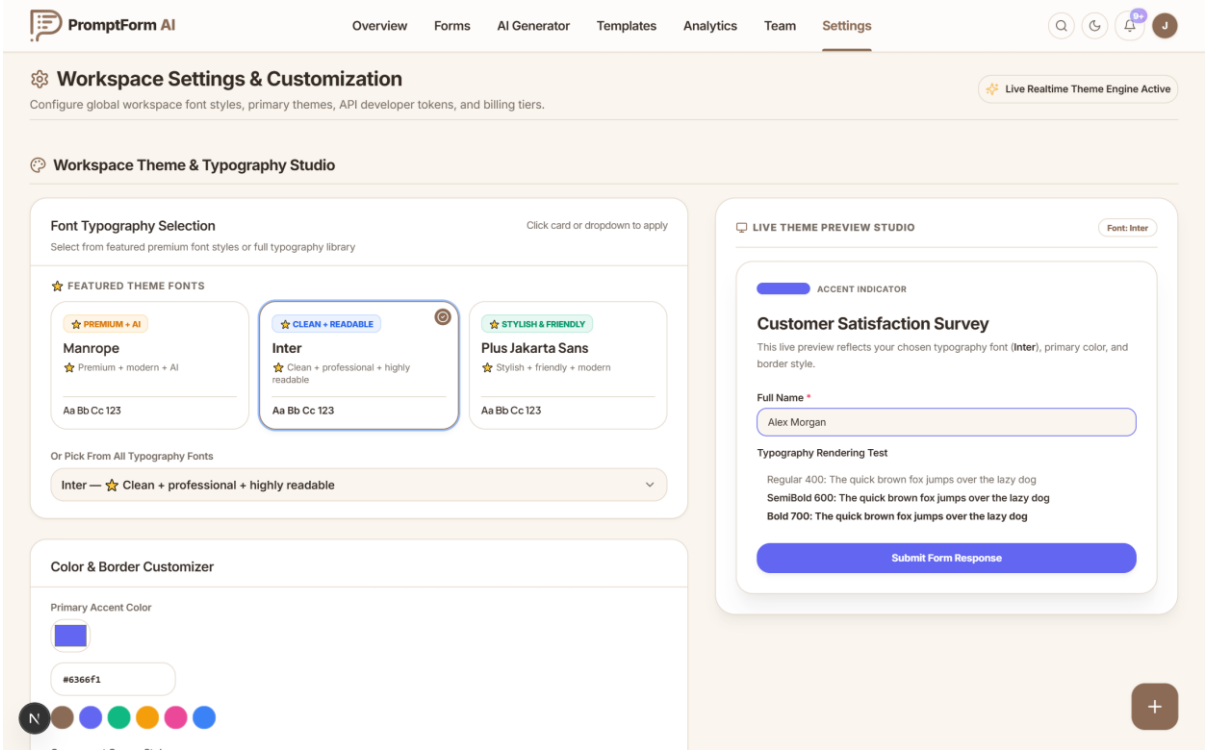


Figure A.8: *Supporting Evidence — Account Settings*

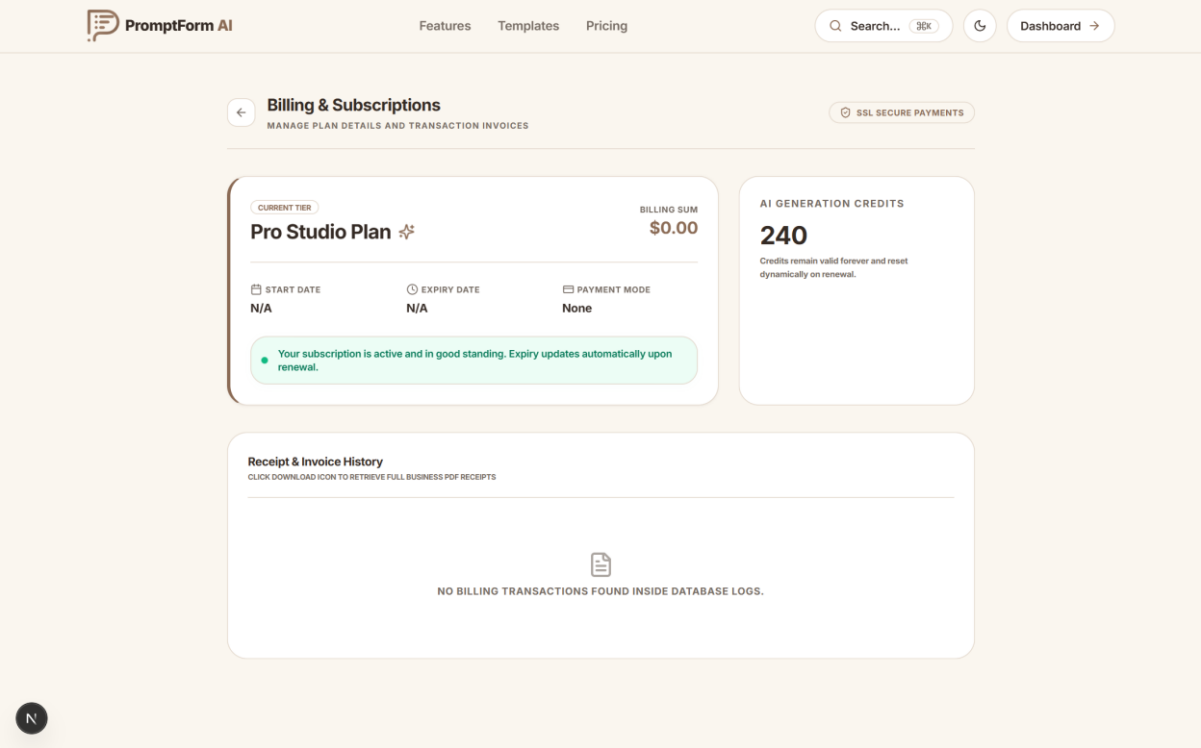


Figure A.9: *Supporting Evidence — Billing and Credits*

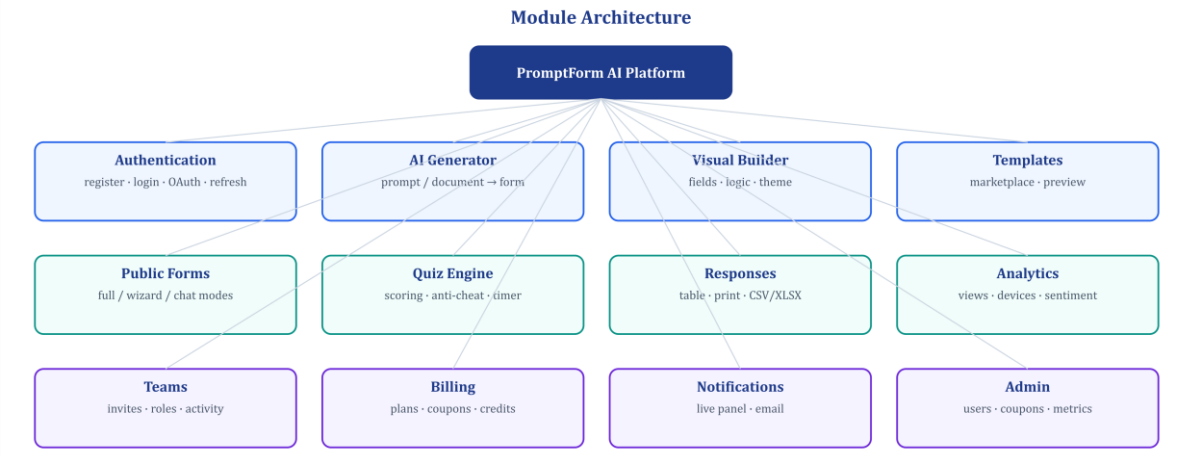


Figure A.10: *Supporting Evidence — Module Architecture*

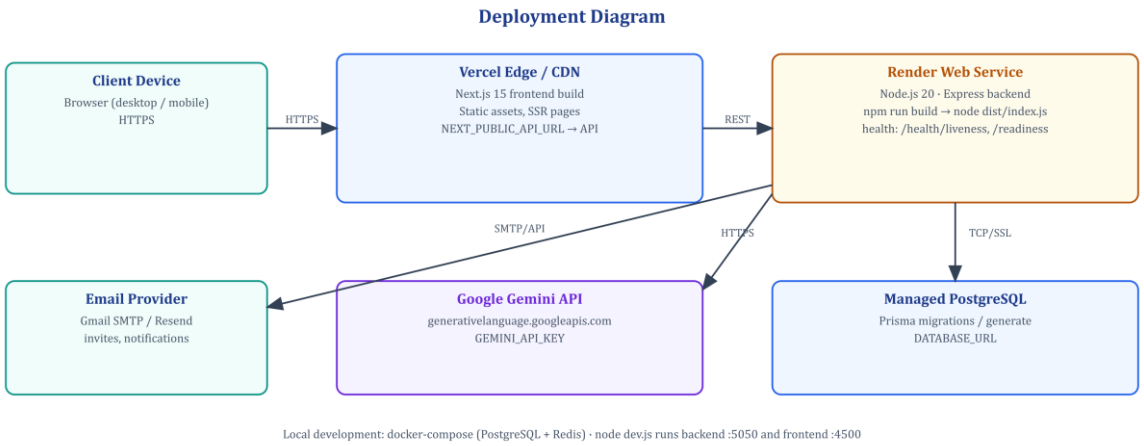


Figure A.11: *Supporting Evidence — Deployment Diagram*

Appendix F – Certificates / Internship Evidence

Attach the internship offer letter, internship completion certificate issued by [COMPANY / ORGANIZATION NAME], attendance / timesheet records and any training certificates on the following pages before hard binding.

Table A.1: *Internship evidence checklist*

Document	Issued By	Attached
Internship offer / joining letter	[COMPANY / ORGANIZATION NAME]	[Yes / No]
Internship completion certificate	[COMPANY / ORGANIZATION NAME]	[Yes / No]
Attendance / timesheet	Company mentor	[Yes / No]
Progress report evaluations (1–4)	Institute mentor	[Yes / No]

Appendix G – Source Code / Repository / Deployment Evidence

Table A.2: *Source code, repository and deployment evidence*

Item	Link / Location	Description	Remarks
Source repository	[Git repository URL]	TypeScript monorepo (main branch, 66 commits)	Private / shared with mentor
Frontend app	https://frontend-tau-nine-n8hxn4thl0.vercel.app	Next.js frontend on Vercel	Deployed
Backend API	https://promptform-api.onrender.com	Express API on Render (render.yaml)	Deployed
Database schema	backend/prisma/schema.prisma	16 Prisma models	—
AI engine	backend/src/lib/aiBrain/	AIFormBrain modules	—
Test suites	backend/src/scripts/runTests.ts	Automated QA runner	50/50 passed

Item	Link / Location	Description	Remarks
QA reports	qa_test_report.json, ai_generator_qa_report.json	Test evidence	—
Local run	node dev.js	Backend :5050, frontend :4500	docker-compose for DB

The complete REST API surface registered by the backend at start-up (73 routes, as reported by the bootstrap sequence) is catalogued below as repository evidence.

Table A.3: *Complete REST API endpoint catalogue*

Method	Endpoint	Purpose	Access
GET	/api/auth/google, /google/callback	Google OAuth sign-in	Public
POST	/api/auth/register	Create account (bcrypt, min. 10 chars)	Public
POST	/api/auth/login	Issue access + refresh token, new session	Public
POST	/api/auth/reset-password	Reset password	Public
GET	/api/auth/me, /profile	Current user profile	JWT
POST	/api/auth/upgrade	Upgrade plan, record transaction	JWT
POST	/api/auth/refresh	Rotate tokens	Refresh token
GET	/api/auth/transactions	Receipts / audit trail	JWT
POST	/api/auth/logout	Invalidate active session	JWT
GET / POST	/api/auth/coupons	List / create coupons	Admin
DELETE	/api/auth/coupons/:id	Delete coupon	Admin
POST	/api/auth/coupons/validate	Validate promo code	JWT
GET / POST	/api/forms	List / create forms	JWT
GET	/api/forms/notifications/live	Live notification feed	JWT
POST	/api/forms/notifications/live/read-all, /:id/read	Mark notifications read	JWT
DELETE	/api/forms/notifications/live, /:id	Clear notifications	JWT
POST	/api/forms/notifications/live/test, /test-email	Send test notification / e-mail	JWT
GET	/api/forms/notifications/live/email-status/:messageId	E-mail delivery status	JWT
GET	/api/forms/collaborations/shared-with-me	Forms shared with user	JWT
GET	/api/forms/:id	Get form (owner or public by share id)	JWT / Public

Method	Endpoint	Purpose	Access
PUT / PATCH	/api/forms/:id	Update form, settings (deep merge)	JWT + role
PUT	/api/forms/:id/questions	Upsert questions (UUIDs preserved)	JWT + role
DELETE	/api/forms/:id	Delete form (cascade)	Owner
GET / POST	/api/forms/:id/collaborators	List / add collaborators	Owner
PUT / DELETE	/api/forms/:id/collaborators/:collaboratorId	Change role / remove	Owner
POST	/api/forms/:id/submit	Public submission (rate limited)	Public
GET	/api/forms/:id/responses	List responses	JWT + role
GET	/api/forms/:id/export	CSV export with scores	JWT + role
POST	/api/ai/generate, /generate-from-file	AI form generation	Optional JWT
POST	/api/ai/analyze-sentiment	Sentiment of text answers	JWT + plan
POST	/api/ai/analyze-response	AI response insights	JWT + plan
POST	/api/ai/chat-stream	Streaming AI assistant	JWT + plan
POST	/api/upload	File upload (documents, answers)	Public / JWT
GET	/api/teams/activity/recent	Recent activity across teams	JWT
GET / POST	/api/teams	List / create teams	JWT
GET	/api/teams/invitations/verify/:token	Verify invite token	Public
POST	/api/teams/invitations/accept/:token	Accept invite (atomic)	JWT
GET	/api/teams/:id, /:id/forms, /:id/activity	Team details, forms, activity	Member
GET / DELETE	/api/teams/:id/invitations[/:inviteId]	List / revoke invitations	Team admin
POST	/api/teams/:id/members	Invite member	Team admin
PUT / DELETE	/api/teams/:id/members/:memberId	Change role / remove member	Team admin
PUT / DELETE	/api/teams/:id	Rename / delete team	Owner
GET	/api/workflows/form/:formId	List workflows of a form	JWT
POST / PUT / DELETE	/api/workflows[/:id]	Manage workflows	JWT
GET	/api/analytics/workspace	Workspace KPIs and trends	JWT
GET	/api/analytics/form/:formId	Form metrics and funnel	JWT
POST	/api/analytics/form/:formId/view, /event	Record view / event	Public
GET / POST	/api/templates[/:id]	Template marketplace	Public / JWT

Method	Endpoint	Purpose	Access
GET	/health, /health/liveness, /health/readiness	Health checks	Public

Selected source excerpts that are referenced in the report are reproduced below; the complete code is available in the repository.

```
for (const model of candidateModels) {          // flash-lite → flash → pro
  const payload = {
    contents: [...history, { role: 'user', parts: currentParts }],
    generationConfig: { responseMimeType: 'application/json',
                        ...(responseSchema ? { responseSchema } : {}) },
    systemInstruction: { parts: [{ text: systemInstruction }] },
  };
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 45000);
  try {
    const res = await fetch(`${GEMINI_URL}/${model}:generateContent?key=${apiKey}`,
      { method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload), signal: controller.signal });
    if (!res.ok) { /* invalid key → stop; inline data rejected → retry text-only;
                     otherwise continue with next model */ continue; }
    return JSON.parse(await extractText(res));
  } finally { clearTimeout(timeoutId); }
}
// all models failed → caller falls back to AIFormBrain.process(prompt)
```

Listing G.1: Gemini call with model fallback chain (backend/src/routes/ai.ts, simplified)

```
const apiLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 1000 });
app.use('/api/', apiLimiter);
const submissionLimiter = rateLimit({ windowMs: 60 * 1000,
  max: process.env.NODE_ENV === 'production' ? 60 : 1000 });
app.use('/api/forms/:id/submit', submissionLimiter);

app.use('/api/auth', authRouter);       app.use('/api/forms', formsRouter);
app.use('/api/ai', aiRouter);           app.use('/api/upload', uploadRouter);
app.use('/api/teams', teamsRouter);     app.use('/api/workflows', workflowsRouter);
app.use('/api/analytics', analyticsRouter);
app.use('/api/templates', templatesRouter);
app.use(errorHandler);
```

Listing G.2: Router mounting and rate limiting (backend/src/index.ts)

```
model Form {
  id            String     @id @default(uuid())
  title         String
  status        String     @default("DRAFT")   // DRAFT | PUBLISHED | CLOSED
  settings      Json       // collect_emails, password, timer_limit, anti_cheat …
  theme         Json       // primary_color, background_color, font_family
  ownerId       String
  owner         User       @relation("UserForms", fields: [ownerId],
                                    references: [id], onDelete: Cascade)
  teamId        String?
  uniqueShareId String?    @unique
  questions     Question[]
  responses     Response[]
  events        FormEvent[]
  @@index([ownerId])
  @@index([teamId])
}
```

Listing G.3: Form model (backend/prisma/schema.prisma, abridged)

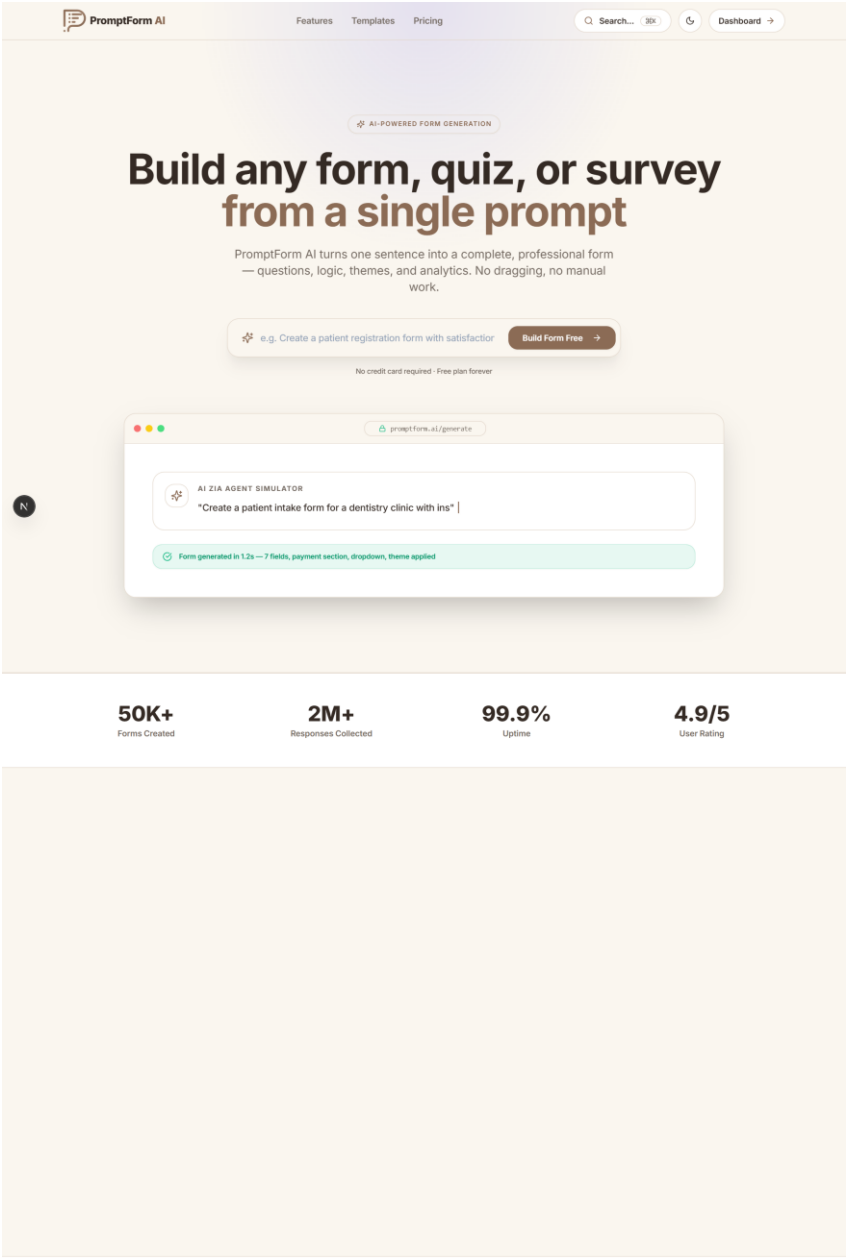


Figure A.12: *Supporting Evidence — Landing Page (full length)*

Appendix H – Mentor / Evaluator Remarks

Table A.4: *Mentor / evaluator remarks*

Evaluator	Remarks	Signature / Date
Mentor (Company) [COMPANY MENTOR NAME]		
Mentor (Institute) [INSTITUTE MENTOR NAME]		
External Evaluator		

CHAPTER B

FINAL HARD-BOUND SUBMISSION CHECKLIST

Table B.1: *Final hard-bound submission checklist*

Check	Requirement	Status / Remarks
☑	Cover page	Included
☑	Certificate	Included – signatures pending
☑	Declaration	Included – signature pending
☑	Acknowledgement	Included
☑	Abstract / Executive Summary	Included with keywords
☑	Table of Contents updated	Page references updated
☑	List of Figures updated	Included
☑	List of Tables updated	Included
☑	All chapters completed	Chapters 1–15
☑	Figures/diagrams captioned and referenced	Chapter-wise numbering
☑	Tables captioned and referenced	Chapter-wise numbering
☑	Testing evidence	Chapter 9, Appendix G
☑	Final results and performance analysis	Chapter 10
☑	APA 7th references	Chapter 15
☑	Appendices/supporting evidence	Appendices A–H
☐	Mentor/Institute evaluation and signatures	To be completed before binding

CHAPTER C

FORMATTING AND SUBMISSION GUIDELINES

- Paper size: A4.
- Body font: Cambria, 12 pt; headings should be consistent.
- Line spacing: 1.5 for body text; single spacing may be used in tables, captions and references where appropriate.
- Margins: approximately 1 inch left/right/top and 0.75 inch bottom; institutional binding rules may override these values.
- Body text should be justified.
- Start each chapter on a new page.
- Number figures and tables chapter-wise, e.g., Figure 6.1 and Table 9.1.
- Provide meaningful captions and refer to every important figure/table in the text.
- Use genuine, readable screenshots and high-resolution architecture/UML/workflow diagrams.
- Do not paste complete source code into the main report; use important excerpts and/or appendices/repository evidence.
- Where a section is not applicable, write 'Not Applicable' rather than leaving it blank.
- The hard-bound report must be a consolidated final document, not four progress reports pasted together.
- Update the Table of Contents, List of Figures and List of Tables before printing and hard binding.
- Ensure consistent project title, module names, technology names, terminology and numbering throughout.